Advanced Representation Learning
(고급표현학습)

Instructor: Donghyeon Cho

The Department of Computer Science

Hanyang University

한양대학교
HANYANG UNIVERSITY

Contents of Table

- 1. Toward Contrastive Learning
- 2. Contrastive Training Objectives
  - Siamese Networks
  - Contrastive Loss
  - Triplet Loss
  - Lifted Structured Loss
  - N-pair Loss
  - NCE
  - InforNCE
  - Soft-Nearest Neighbors Loss
- 3. Common Setup for Contrastive Learning

Toward Contrastive Learning

From Pretext Tasks to Contrastive Learning

- They are all handcrafted Objectives

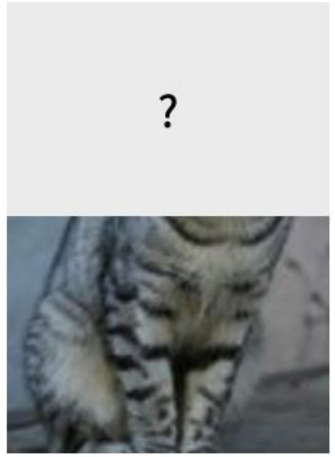
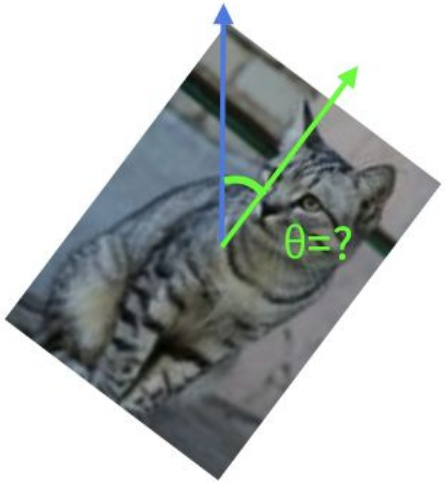


image completion



rotation prediction



“jigsaw puzzle”

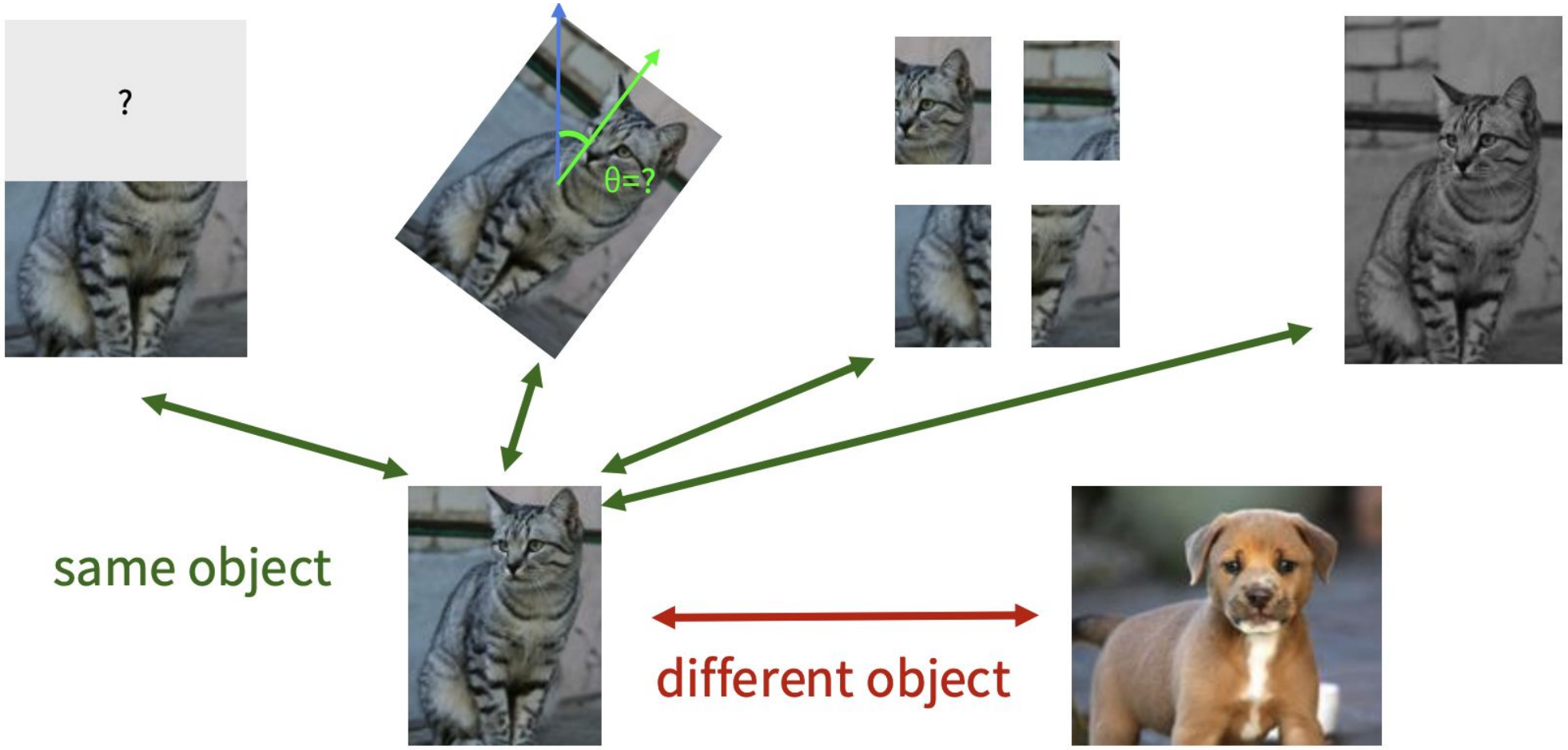


colorization

Can we learn representations without designing a task?

A More General Pretext Task?

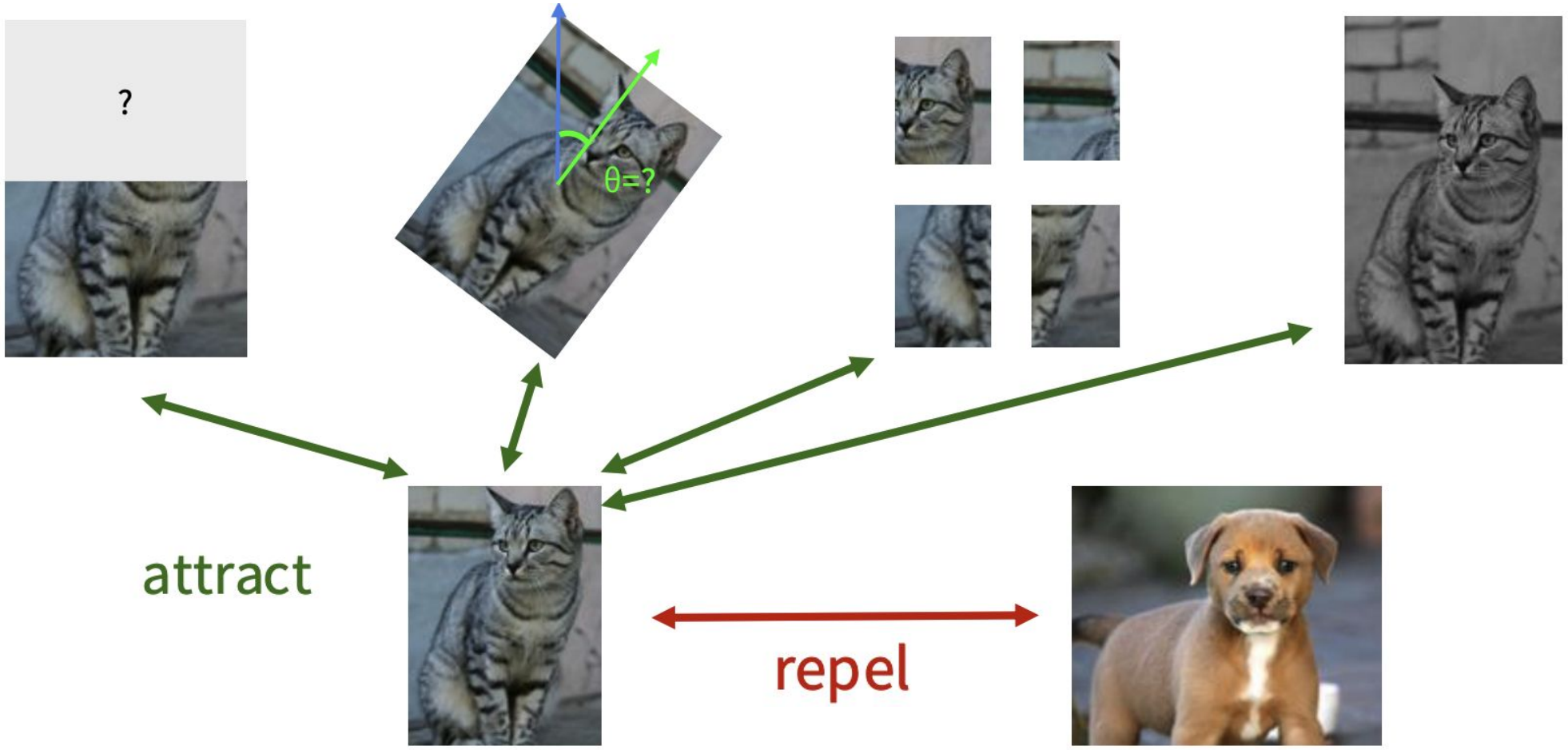
- Good representations place semantically similar samples nearby.



Representation Learning = Learning Similarity

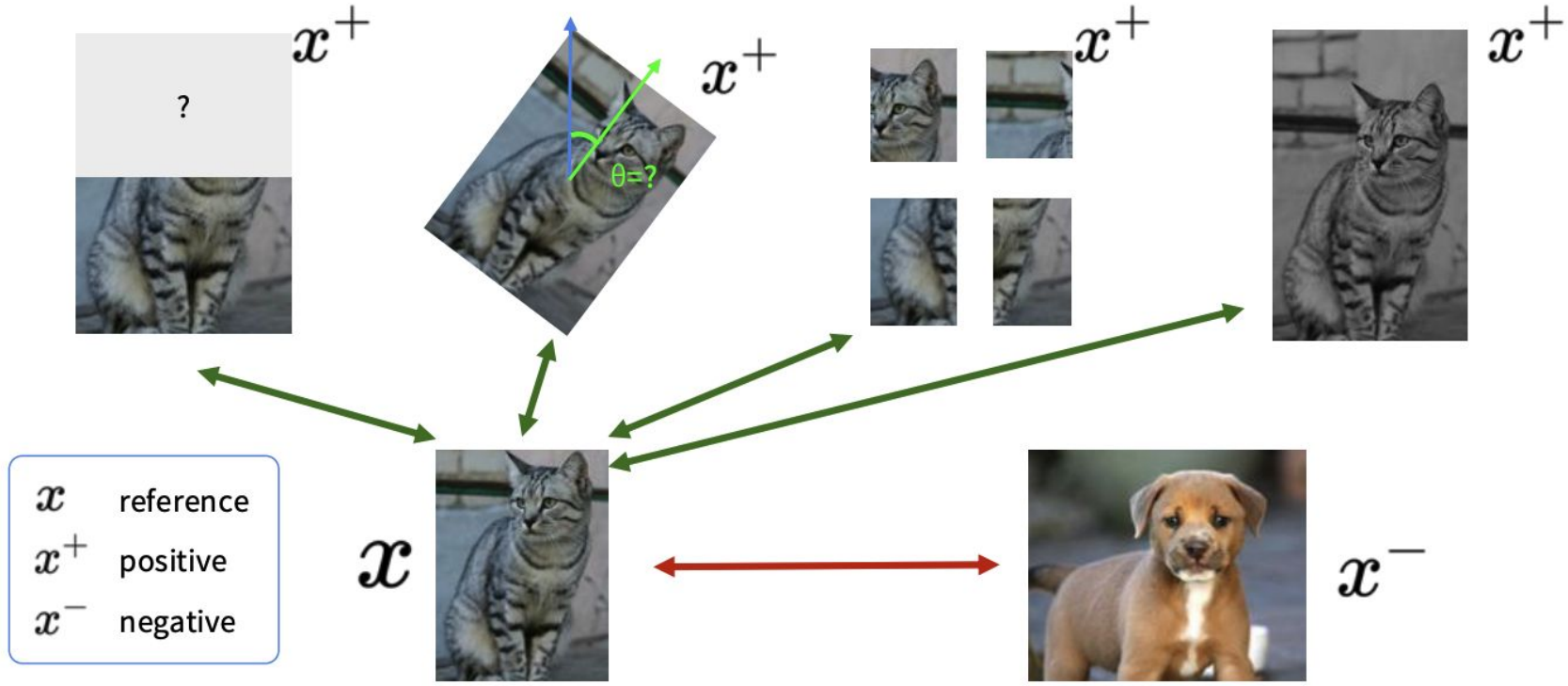
Similarity as Supervision

- Similarity provides a learning signal.



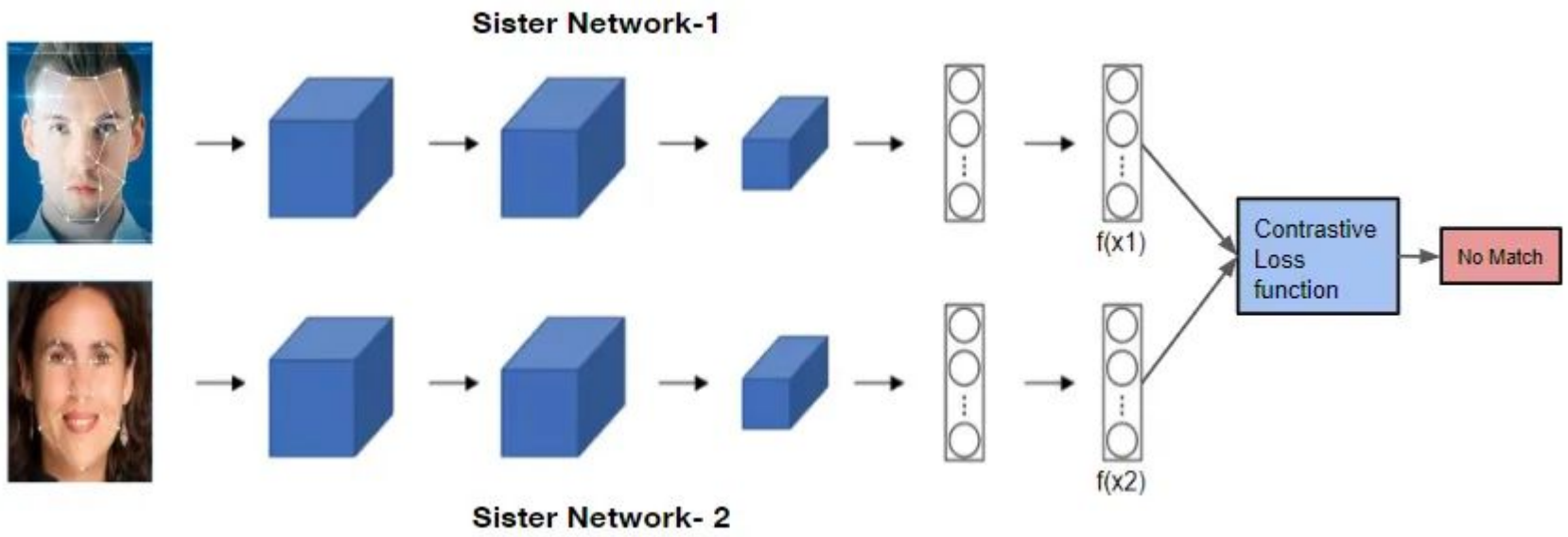
Metric Learning

- Pull Similar Samples Together
- Push Dissimilar Samples Apart



Siamese Network

- Both images are mapped into the same embedding space.



Same Network + Shared Weights

z1=f(x1), z2=f(x2)

If they are the same person decreases, d(z1,z2), otherwise d(z1,z2) increases .

Contrastive Representation Learning

- Learn embedding space
  - Similar sample pairs close to each other
  - Dissimilar ones are far apart
- Can be applied to both supervised and unsupervised settings.
- What we want:

$$\text{score}(f(x), f(x^+)) >> \text{score}(f(x), f(x^-))$$

x: reference sample; x^+ positive sample; x^- negative sample

Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-).

Contrastive Training Objectives

Contrastive Loss

Intuitive Goal

Learn an encoder function $f_\theta(x)$ that maps input samples into an embedding space where:

Positive Pairs **Minimize distance**

Samples from the **same class** are pulled close together.

Negative Pairs **Maximize distance**

Samples from **different classes** are pushed apart beyond a margin ε.

Mathematical Formulation

Contrastive Loss Function

$$L_{cont} = 1_{[y_i=y_j]} \|f_\theta(x_i) - f_\theta(x_j)\|_2^2 + 1_{[y_i\neq y_j]} \max(0, \varepsilon - \|f_\theta(x_i) - f_\theta(x_j)\|_2)^2$$

1. Equivalent Class Case ($y_i = y_j$)

$$L = \|f_\theta(x_i) - f_\theta(x_j)\|_2^2$$

2. Different Class Case ($y_i \neq y_j$)

$$L = \max(0, \varepsilon - \|f_\theta(x_i) - f_\theta(x_j)\|_2)^2$$

where ε (margin) defines the minimum distance required between different classes.

Contrastive Loss considers only pairs

- Many negatives become inactive
- Does not explicitly enforce relative ordering

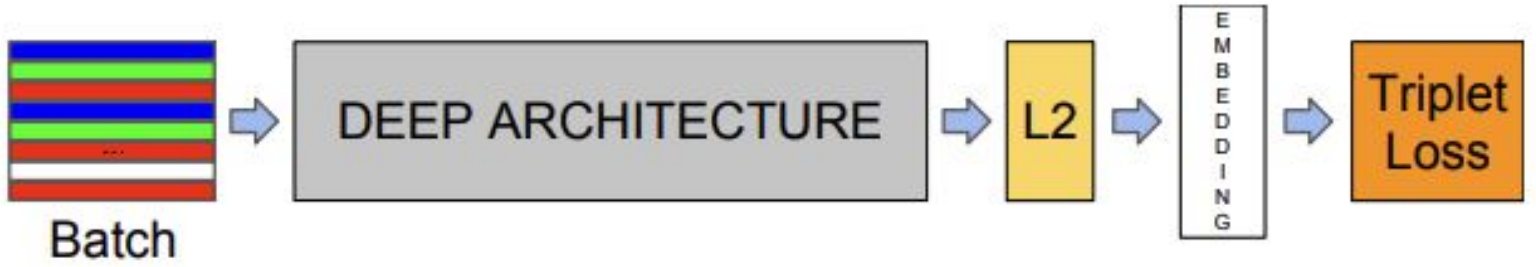
Chopra et al., 2005, Learning a Similarity Metric Discriminatively, with Application to Face Verification

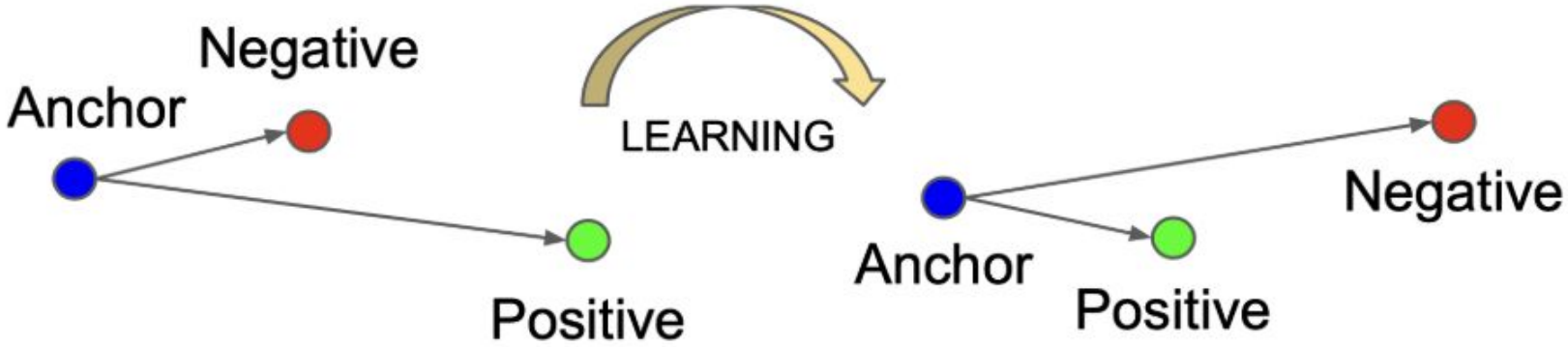
Triplet Loss

- [FaceNet] learn face recognition of the same person at different poses and angles.
  - Triplet Loss uses Anchor–Positive–Negative triplets
    - Enforces relative similarity
    - Focuses on ranking relationships









Schroff et al., 2015, FaceNet: A Unified Embedding for Face Recognition and Clustering

Triplet Loss

Intuitive Goal

Map samples into an embedding space using triplets: Anchor (x), Positive (x^+), and Negative (x^-).

POSITIVE **Minimize Distance**

Anchor and positive samples (same class) are pulled close together.

NEGATIVE **Maximize Distance**

Anchor and negative samples (different classes) are pushed apart.

Key: Selecting challenging negative samples (hard negatives) is crucial to truly improve the model.

Mathematical Formulation

Triplet Loss Function

$$L_{\text{triplet}} = \sum max(0, ||f(x) - f(x^+)||_2^2 - ||f(x) - f(x^-)||_2^2 + \varepsilon)$$

Margin Parameter (ε)

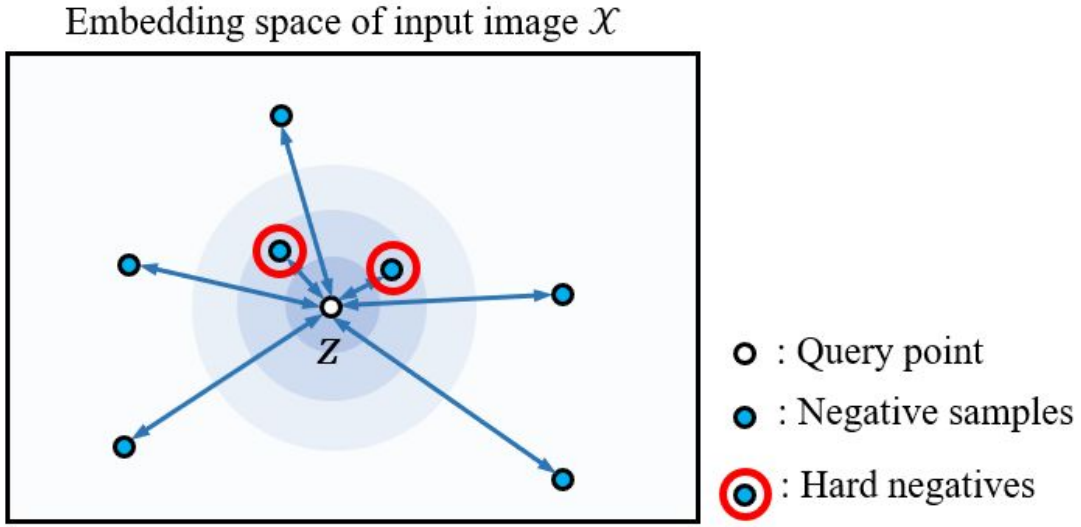
The margin **ε** is configured as the minimum offset required between the distances of similar (anchor-positive) vs dissimilar (anchor-negative) pairs.

Problem 1: Triplet Explosion
- Possible triplet number is N^3 for N samples

Problem 2: Most Triplets are Useless
- Easy triplets dominate

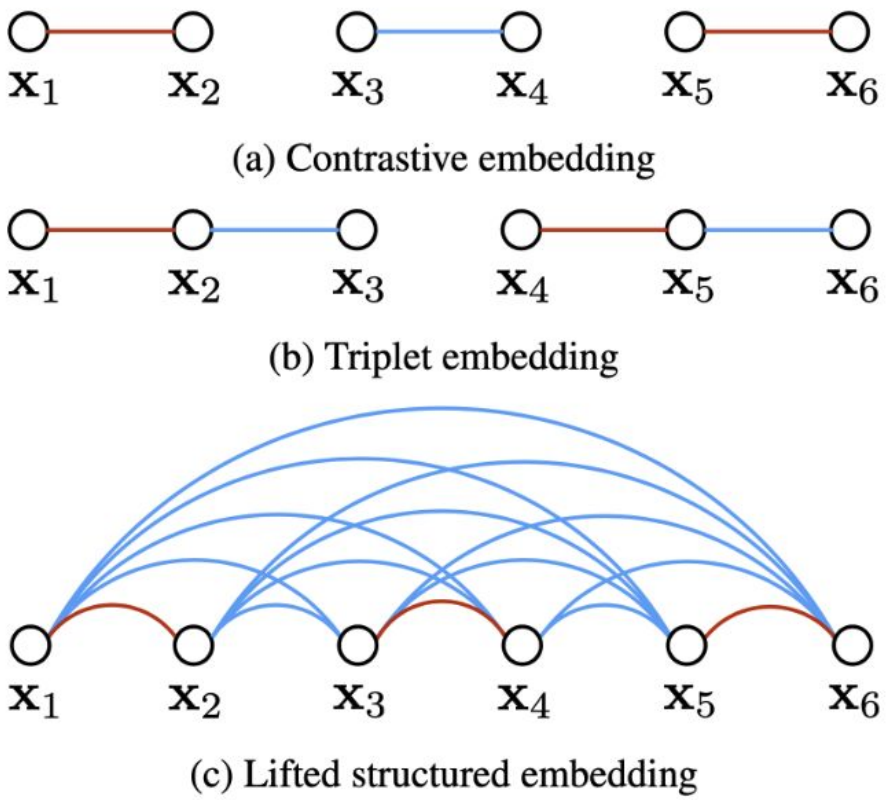
Hard Negative Mining

- Most negative samples are already far from the anchor.
  - Easy negatives contribute little to the loss and gradients.
- Hard negatives are close to the anchor but belong to different classes.
  - Hard negatives provide stronger learning signals.
  - Selecting informative negatives improves representation learning.
  - Hard Negative Mining focuses training on the most challenging samples.



Lifted Structured Loss

- Lifted Structured Loss utilizes all the pairwise edges within one training batch for better computational efficiency.



(a) Contrastive embedding

(b) Triplet embedding

(c) Lifted structured embedding

Red and blue edges connect similar and dissimilar sample pairs respectively.

$$\mathcal{L}_{\text{struct}} = \frac{1}{2|\mathcal{P}|} \sum_{(i,j)\in\mathcal{P}} \max(0, \mathcal{L}_{\text{struct}}^{(ij)})^2$$

$$\text{where } \mathcal{L}_{\text{struct}}^{(ij)} = D_{ij} + \max\Big(\max_{(i,k)\in\mathcal{N}} \epsilon - D_{ik}, \max_{(j,l)\in\mathcal{N}} \epsilon - D_{jl} \Big)$$

$$D_{ij} = |f(\mathbf{x}_i) - f(\mathbf{x}_j)|_2$$

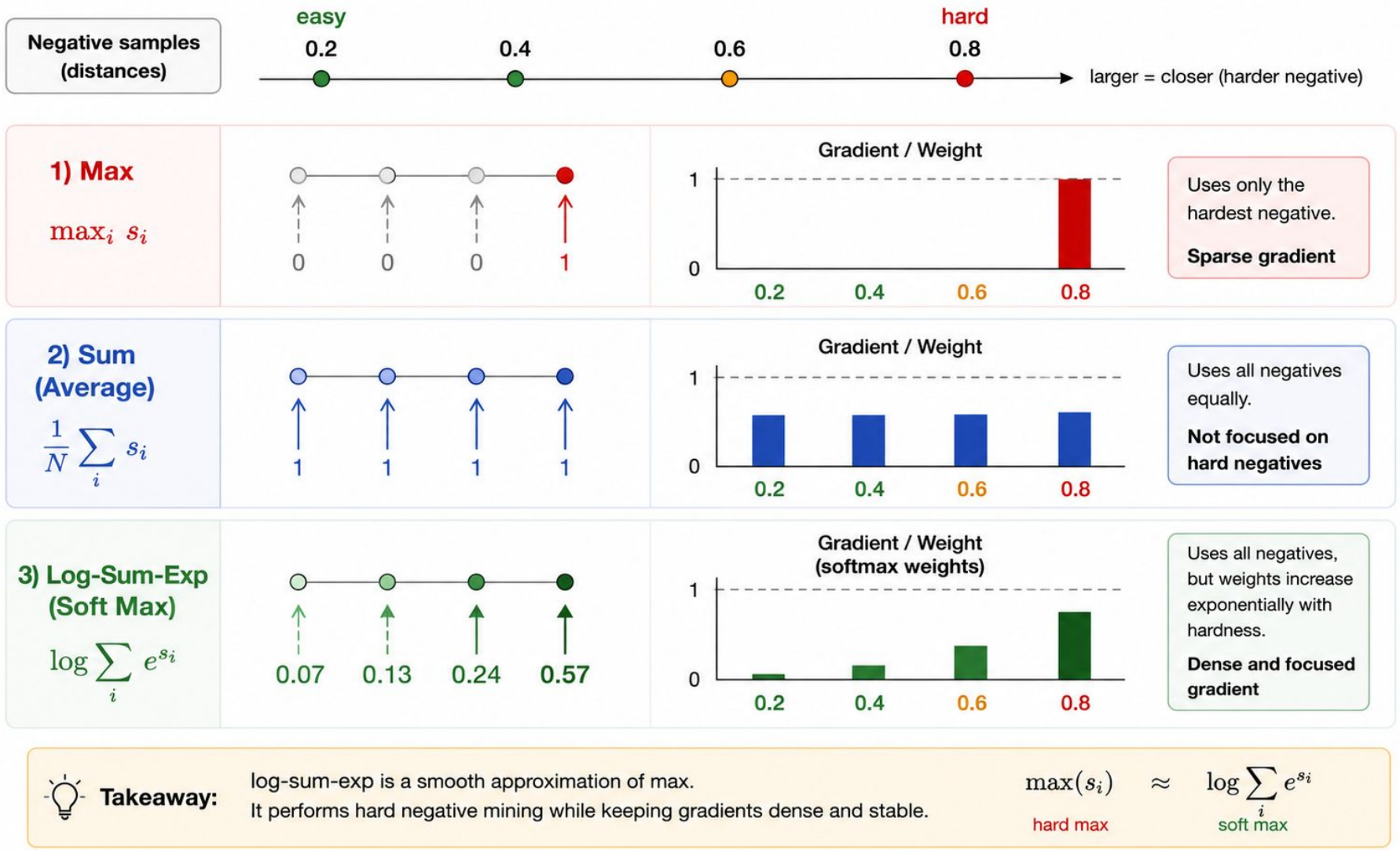
- *P* contains the set of positive pairs
- *N* is the set of negative pairs

not smooth, so relaxed to be

$$\mathcal{L}_{\text{struct}}^{(ij)} = D_{ij} + \log\Big(\sum_{(i,k)\in\mathcal{N}} \exp(\epsilon - D_{ik}) + \sum_{(j,l)\in\mathcal{N}} \exp(\epsilon - D_{jl}) \Big)$$

Song et al., 2015, Deep Metric Learning via Lifted Structured Feature Embedding

Why Replace Max?



Lifted Structured Loss

- Limitations of Lifted Structured Loss
  - Requires dense pairwise comparisons (O(N^2))
  - Still fundamentally pair-based
    - Optimize pair relations instead of directly comparing multiple classes
  - Does not fully exploit multi-class batch structure
    - 1 positive <-> 1 negative
  - Optimization becomes increasingly complex
- Can we compare one positive against all negatives simultaneously?
- → N-pair Loss (NIPS 2016)
  - 1 positive <-> many negatives
  - → More information
  - → No need hard negative mining

N-pair Loss

- Multi-Class N-pair loss generalizes triplet loss to include comparison with multiple negative samples.
- Given a $(N+1)$-tuplet of training samples, $\{\mathbf{x}, \mathbf{x}^+, \mathbf{x}_1^-, \dots, \mathbf{x}_{N-1}^-\}$, including one positive and $N-1$ negative ones, N-pair loss is defined as:

$$\mathcal{L}_{\text{N-pair}}(\mathbf{x}, \mathbf{x}^+, \{\mathbf{x}_i^-\}_{i=1}^{N-1}) = \log\left(1 + \sum_{i=1}^{N-1} \exp(f(\mathbf{x})^\top f(\mathbf{x}_i^-) - f(\mathbf{x})^\top f(\mathbf{x}^+))\right)$$

$$= -\log \frac{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+))}{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+)) + \sum_{i=1}^{N-1} \exp(f(\mathbf{x})^\top f(\mathbf{x}_i^-))}$$

If we only sample one negative sample per class, it is equivalent to the softmax loss for multi-class classification.

Sohn et al., NeurIPS 2016, Improved Deep Metric Learning with Multi-class N-pair Loss Objective
https://lilianweng.github.io/

N-pair Loss

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+))}{\exp(s(f(x), f(x^+)) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-))} \right]$$



x



x^+



x



x_1^-



x_2^-



x_3^-

...

N-pair Loss

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+))}{\exp(s(f(x), f(x^+)) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-))} \right]$$

score for the positive pair

score for the N-1 negative pairs

This seems familiar …

Cross entropy loss for a N-way softmax classifier!

I.e., learn to find the positive sample from the N samples

Are Negative Samples Just Noise?

- Can we treat negatives as noise?
- Can we interpret it probabilistically?
  - Yes.
  - Positive vs Negative → Data vs Noise
  - The idea is to run logistic regression to tell apart the target data from noise.
  - → Noise Contrastive Estimation (NCE), is a method for estimating parameters of a statistical model

NCE

- Noise Contrastive Estimation (NCE) is a method for estimating parameters of a statistical model.

Let $\mathbf{x}$ be the target sample $\sim P(\mathbf{x}|C=1;\theta) = p_\theta(\mathbf{x})$ and $\tilde{\mathbf{x}}$ be the noise sample $\sim P(\tilde{\mathbf{x}}|C=0) = q(\tilde{\mathbf{x}})$. Note that the logistic regression models the logit (i.e. log-odds) and in this case we would like to model the logit of a sample u from the target data distribution instead of the noise distribution:

$$\ell_\theta(\mathbf{u}) = \log \frac{p_\theta(\mathbf{u})}{q(\mathbf{u})} = \log p_\theta(\mathbf{u}) - \log q(\mathbf{u})$$

After converting logits into probabilities with sigmoid $\sigma(.)$, we can apply cross entropy loss:

$$\mathcal{L}_{\text{NCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[\log \sigma(\ell_\theta(\mathbf{x}_i)) + \log(1-\sigma(\ell_\theta(\tilde{\mathbf{x}}_i)))\right]$$

$$\text{where } \sigma(\ell) = \frac{1}{1+\exp(-\ell)} = \frac{p_\theta}{p_\theta + q}$$

Gutmann et al., PMLR 2010, Noise-contrastive estimation: A new estimation principle for unnormalized statistical models
https://lilianweng.github.io/

InfoNCE

- The InfoNCE loss uses **categorical cross-entropy loss** to identify the positive sample amongst a set of unrelated noise samples.

Given a context vector $\mathbf{c}$, the positive sample should be drawn from the conditional distribution $p(\mathbf{x}|\mathbf{c})$, while $N-1$ negative samples are drawn from the proposal distribution $p(\mathbf{x})$, independent from the context $\mathbf{c}$. For brevity, let us label all the samples as $X = \{\mathbf{x}_i\}_{i=1}^N$ among which only one of them $\mathbf{x}_{\text{pos}}$ is a positive sample. The probability of we detecting the positive sample correctly is:

Bayes Rule

Likelihood Prior

$$P(C = \text{pos} \mid X, c) = \frac{P(X \mid C = \text{pos}, c)P(C = \text{pos})}{\sum_{k=1}^{N} P(X \mid C = k, c)P(C = k)}$$

Marginal

Likelihood

$$P(X \mid C = \text{pos}, c) = p(x_{\text{pos}} \mid c) \prod_{i \neq \text{pos}} p(x_i)$$

Oord et al., 2018, Representation Learning with Contrastive Predictive Coding
https://lilianweng.github.io/

InfoNCE

- The InfoNCE loss uses **categorical cross-entropy loss** to identify the positive sample amongst a set of unrelated noise samples.

Given a context vector $\mathbf{c}$, the positive sample should be drawn from the conditional distribution $p(\mathbf{x}|\mathbf{c})$, while $N-1$ negative samples are drawn from the proposal distribution $p(\mathbf{x})$, independent from the context $\mathbf{c}$. For brevity, let us label all the samples as $X = \{\mathbf{x}_i\}_{i=1}^N$ among which only one of them $\mathbf{x}_{\text{pos}}$ is a positive sample. The probability of we detecting the positive sample correctly is:

divide $\prod_i p(x_i)$

$$p(C = \text{pos}|X, \mathbf{c}) = \frac{p(x_{\text{pos}}|\mathbf{c}) \prod_{i=1,\ldots,N;i\neq\text{pos}} p(\mathbf{x}_i)}{\sum_{j=1}^N \left[p(\mathbf{x}_j|\mathbf{c}) \prod_{i=1,\ldots,N;i\neq j} p(\mathbf{x}_i)\right]} = \frac{\frac{p(\mathbf{x}_{\text{pos}}|c)}{p(\mathbf{x}_{\text{pos}})}}{\sum_{j=1}^N \frac{p(\mathbf{x}_j|\mathbf{c})}{p(\mathbf{x}_j)}} = \frac{f(\mathbf{x}_{\text{pos}}, \mathbf{c})}{\sum_{j=1}^N f(\mathbf{x}_j, \mathbf{c})}$$

where the scoring function is $f(\mathbf{x}, \mathbf{c}) \propto \frac{p(\mathbf{x}|\mathbf{c})}{p(\mathbf{x})}$.

Oord et al., 2018, Representation Learning with Contrastive Predictive Coding
https://lilianweng.github.io/

InfoNCE

- The InfoNCE loss uses **categorical cross-entropy loss** to identify the positive sample amongst a set of unrelated noise samples.

The InfoNCE loss optimizes the negative log probability of classifying the positive sample correctly:

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}\left[\log \frac{f(\mathbf{x}, \mathbf{c})}{\sum_{\mathbf{x}' \in X} f(\mathbf{x}', \mathbf{c})}\right]$$

Oord et al., 2018, Representation Learning with Contrastive Predictive Coding
https://lilianweng.github.io/

N-pair Loss

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+))}{\exp(s(f(x), f(x^+)) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-))} \right]$$

score for the positive pair

score for the N-1 negative pairs

This seems familiar …

Cross entropy loss for a N-way softmax classifier!

I.e., learn to find the positive sample from the N samples

Anchor (x) can be regarded as context (c)

InfoNCE

- Mutual information optimization

$$I(\mathbf{x};\mathbf{c}) = \sum_{\mathbf{x},\mathbf{c}} p(\mathbf{x},\mathbf{c}) \log \frac{p(\mathbf{x},\mathbf{c})}{p(\mathbf{x})p(\mathbf{c})}$$

Oord et al., 2018, Representation Learning with Contrastive Predictive Coding
https://lilianweng.github.io/

InfoNCE

- Mutual information optimization

$$I(\mathbf{x};\mathbf{c}) = \sum_{\mathbf{x},\mathbf{c}} p(\mathbf{x},\mathbf{c}) \log \frac{p(\mathbf{x},\mathbf{c})}{p(\mathbf{x})p(\mathbf{c})} = \sum_{\mathbf{x},\mathbf{c}} p(\mathbf{x},\mathbf{c}) \log \frac{p(\mathbf{x}|\mathbf{c})}{p(\mathbf{x})}$$

$$f(x, c)$$

Learning a good f(x,c)

→ Estimating a good density ratio

→ Increasing Mutual Information

InfoNCE is not simply learned to match positives,

but rather learned in a direction that increases the shared information between x and c.

Oord et al., 2018, Representation Learning with Contrastive Predictive Coding
https://lilianweng.github.io/

Soft-Nearest Neighbors Loss

- Soft-Nearest Neighbors Loss extends it to include **multiple positive samples**.

Given a batch of samples, $\{\mathbf{x}_i, y_i)\}_{i=1}^{B}$ where y_i is the class label of $\mathbf{x}_i$ and a function $f(.,.)$ for measuring similarity between two inputs, the soft nearest neighbor loss at temperature τ is defined as:

Neighbors of the same class

$$\mathcal{L}_{\text{snn}} = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\sum_{i\neq j, y_i=y_j, j=1,\ldots,B}\exp(-f(\mathbf{x}_i,\mathbf{x}_j)/\tau)}{\sum_{i\neq k, k=1,\ldots,B}\exp(-f(\mathbf{x}_i,\mathbf{x}_k)/\tau)}$$

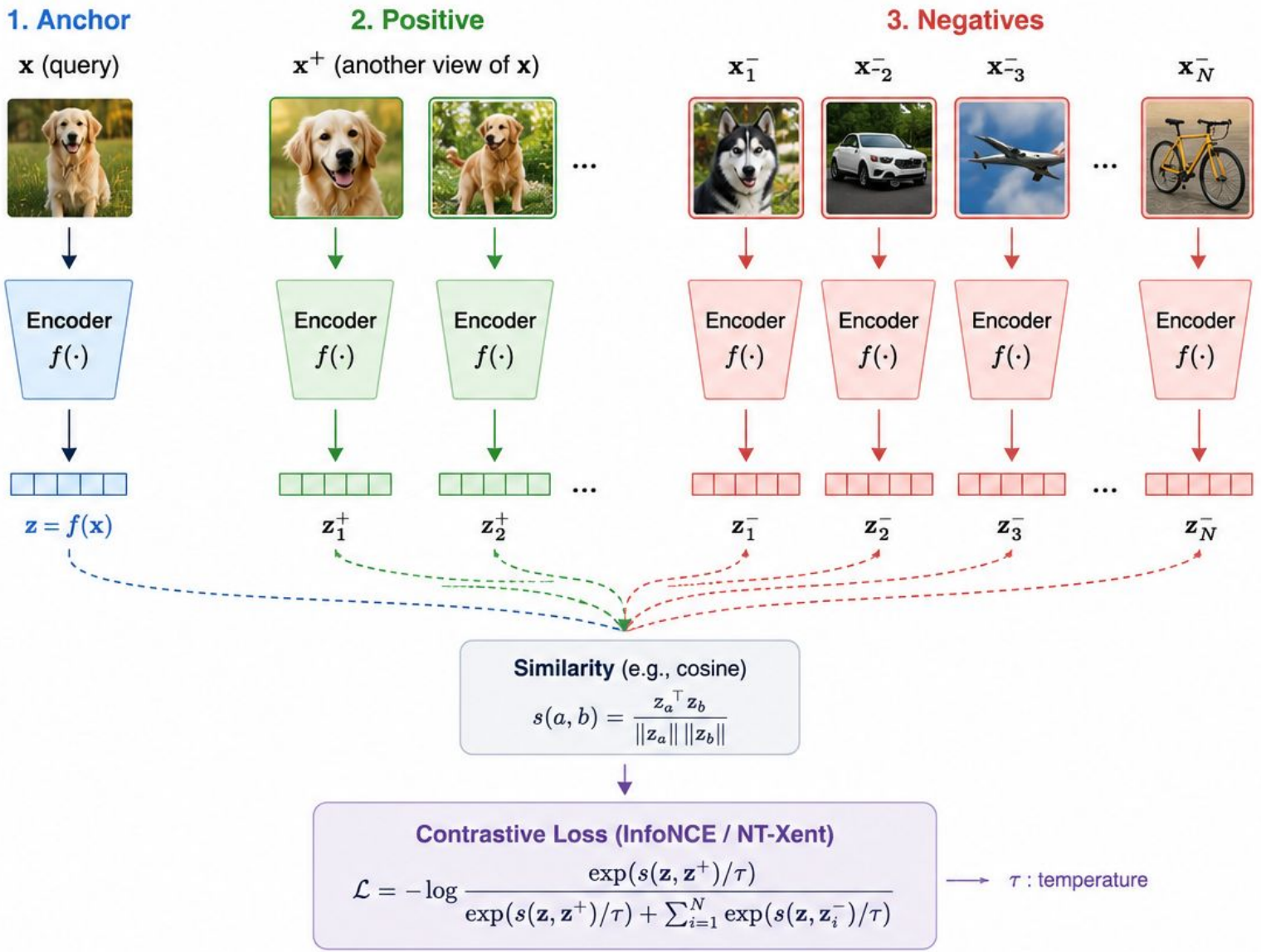
All neighbors

The temperature is used for tuning how concentrated the features are in the representation space.

Salakhutdinov et al., 2007, Learning a Nonlinear Embedding by Preserving Class Neighbourhood Structure
Frosst et al., 2019, Analyzing and Improving Representations with the Soft Nearest Neighbor Loss

Common Setup for Contrastive Learning

Common Setup for Modern Contrastive Learning



Key Components

1. Anchor (Query)

- A sample $\mathbf{x}$ from the dataset.

2. Positive(s)

- Different augmented views of the same sample $\mathbf{x}$.

3. Negatives

- Other samples in the batch (treated as negatives).

4. Encoder $f(\cdot)$

- Maps inputs to representations (often L2-normalized).

5. Similarity

- Measures closeness between representations.

6. Contrastive Loss

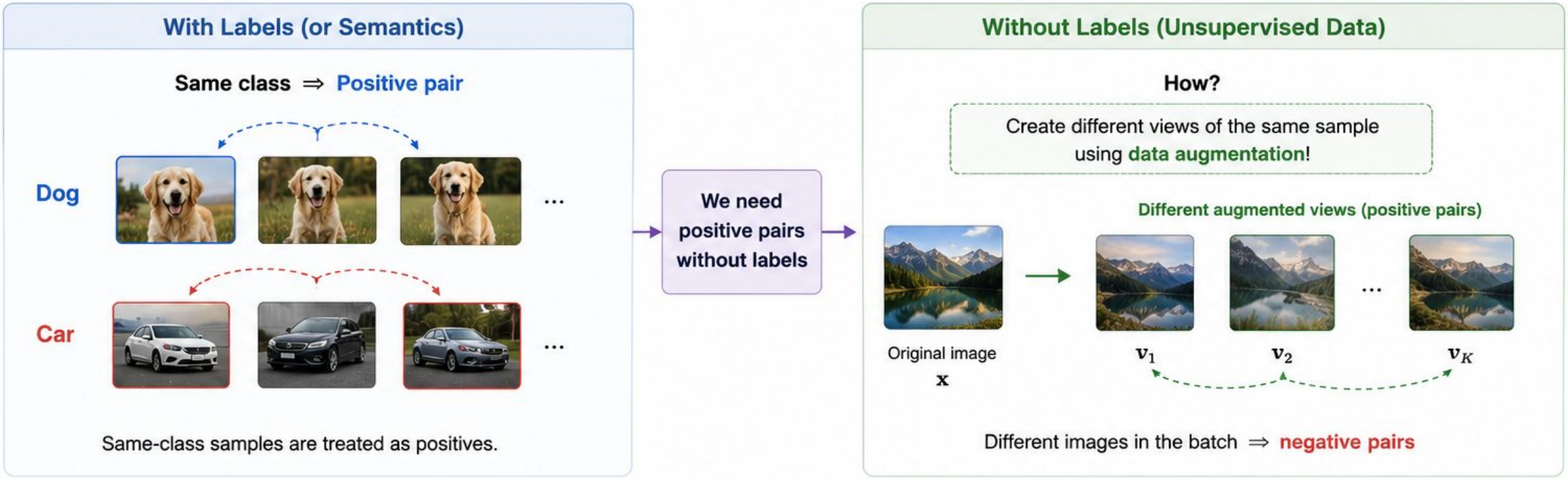
- Pulls positives together and pushes negatives apart.

Legend

- Anchor path
- Positive path
- Negative path
- Similarity computation

Positive Pair Construction

- Contrastive learning incorporates multiple positive and negative samples.



- Symmetry: $\forall \mathbf{x}, \mathbf{x}^+, p_{\text{pos}}(\mathbf{x}, \mathbf{x}^+) = p_{\text{pos}}(\mathbf{x}^+, \mathbf{x})$
- Matching marginal: $\forall \mathbf{x}, \int p_{\text{pos}}(\mathbf{x}, \mathbf{x}^+) d\mathbf{x}^+ = p_{\text{data}}(\mathbf{x})$

Are positive and negative pairs always constructed correctly?

Check for updates

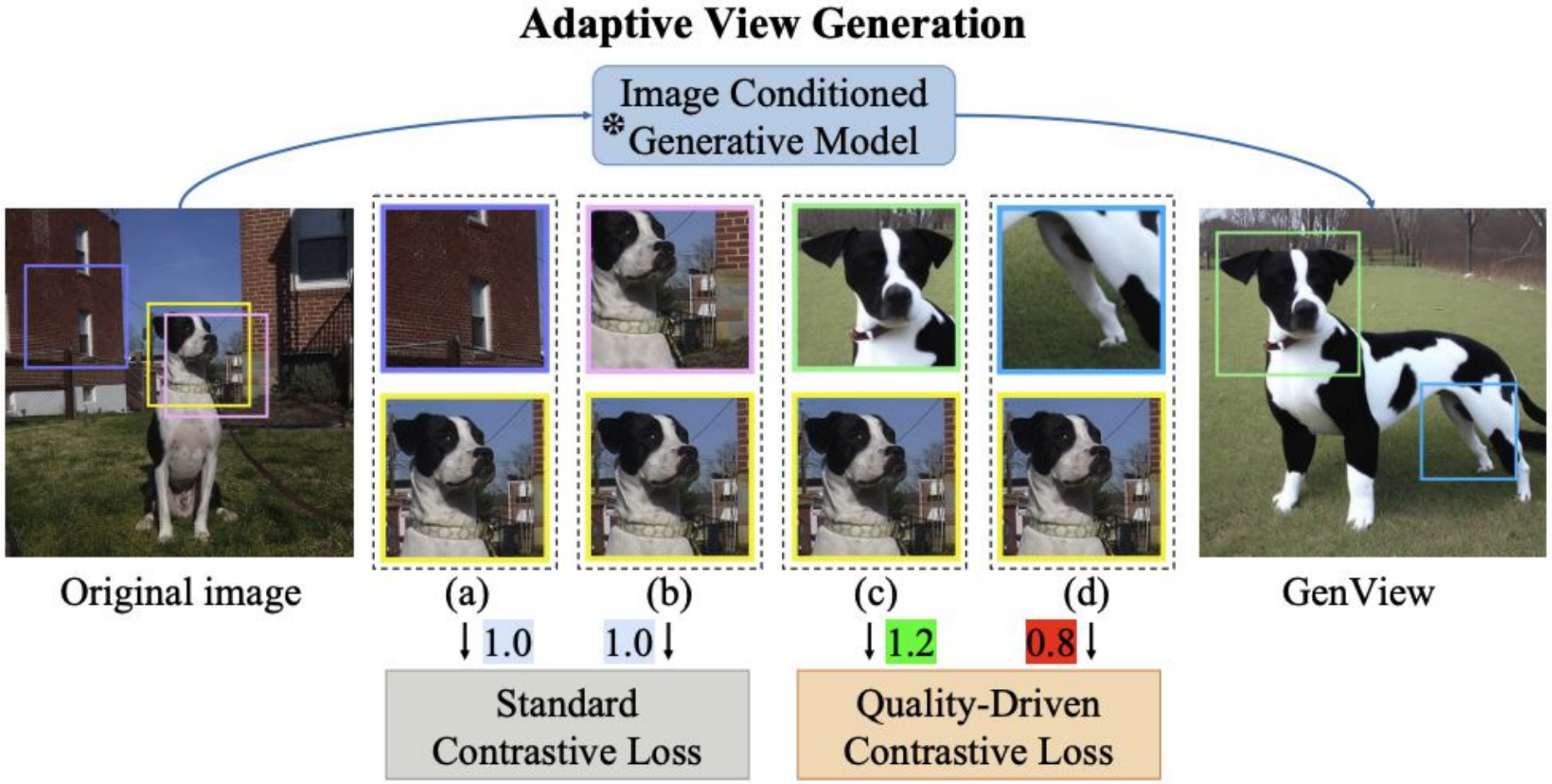
GenView: Enhancing View Quality with Pretrained Generative Model for Self-Supervised Learning

Xiaojie Li[1,2], Yibo Yang[3(✉)], Xiangtai Li[4], Jianlong Wu[1(✉)], Yue Yu[2], Bernard Ghanem[3], and Min Zhang[1]

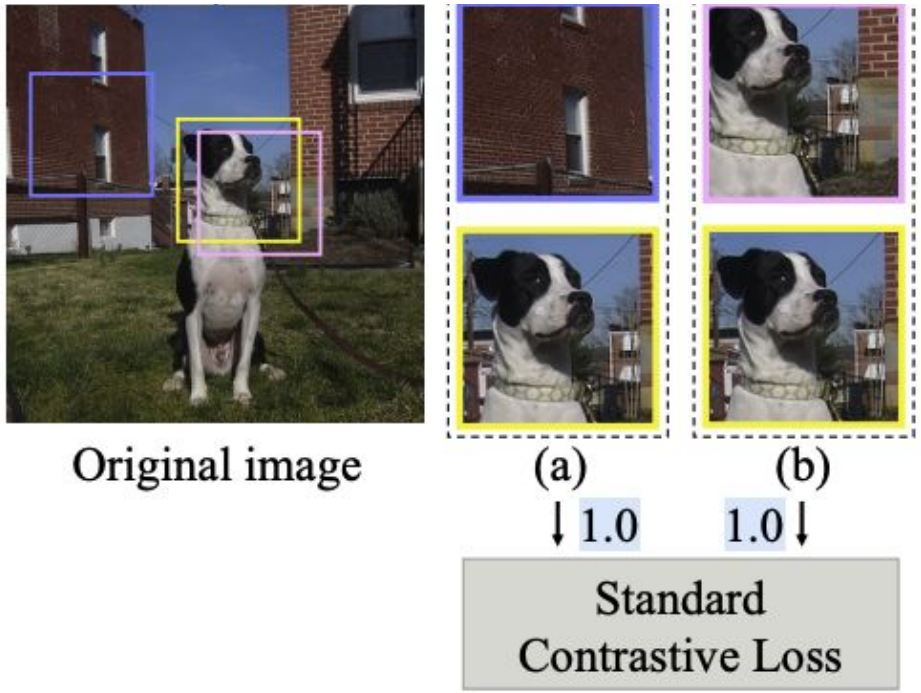
[1] Harbin Institute of Technology (Shenzhen), Shenzhen, China
{wujianlong,zhangmin2021}@hit.edu.cn
[2] Peng Cheng Laboratory, Shenzhen, China
yuy@pcl.ac.cn
[3] King Abdullah University of Science and Technology (KAUST), Thuwal, Saudi Arabia
yibo.yang93@gmail.com, bernard.ghanem@kaust.edu.sa
[4] S-Lab, Nanyang Technological University, Singapore, Singapore
https://github.com/xiaojieli0903/genview

Li et al., ECCV 2024, GenView: Enhancing View Quality with Pretrained Generative Model for Self-Supervised Learning

Can a Generative Model Create Better Positive Views?



Limitations of Handcrafted Views



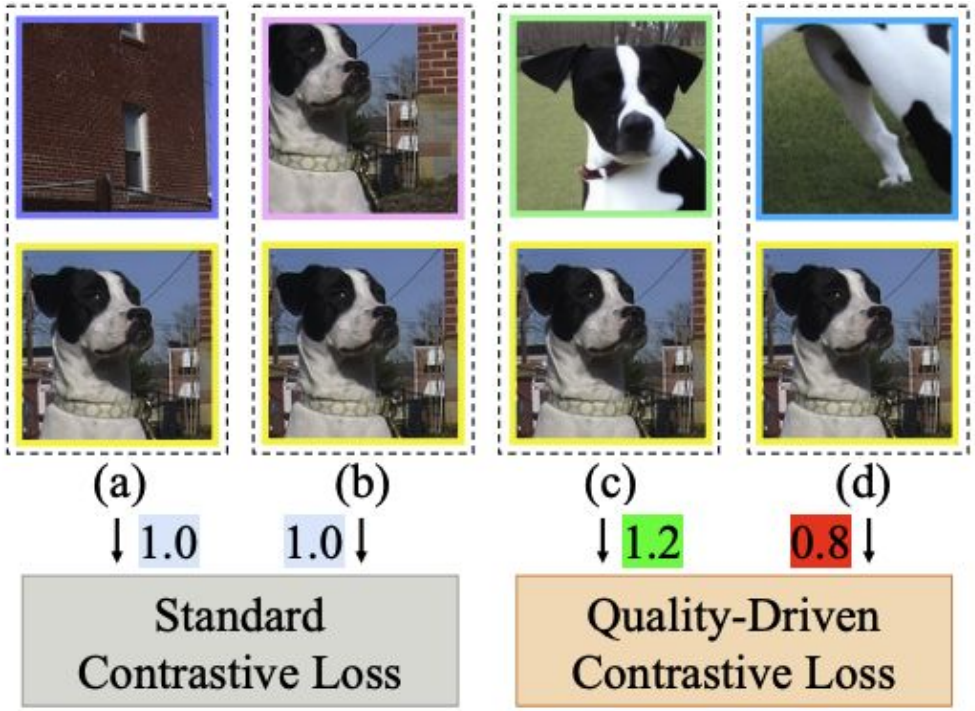
Standard augmentations have two limitations

(a) **False Positive Pair**: A strong crop may remove the main object.

(b) **Limited Diversity**: A weak crop may produce two nearly identical views.

A good positive pair should preserve semantics while introducing diversity

With Generative Model..



Generative model can overcome..

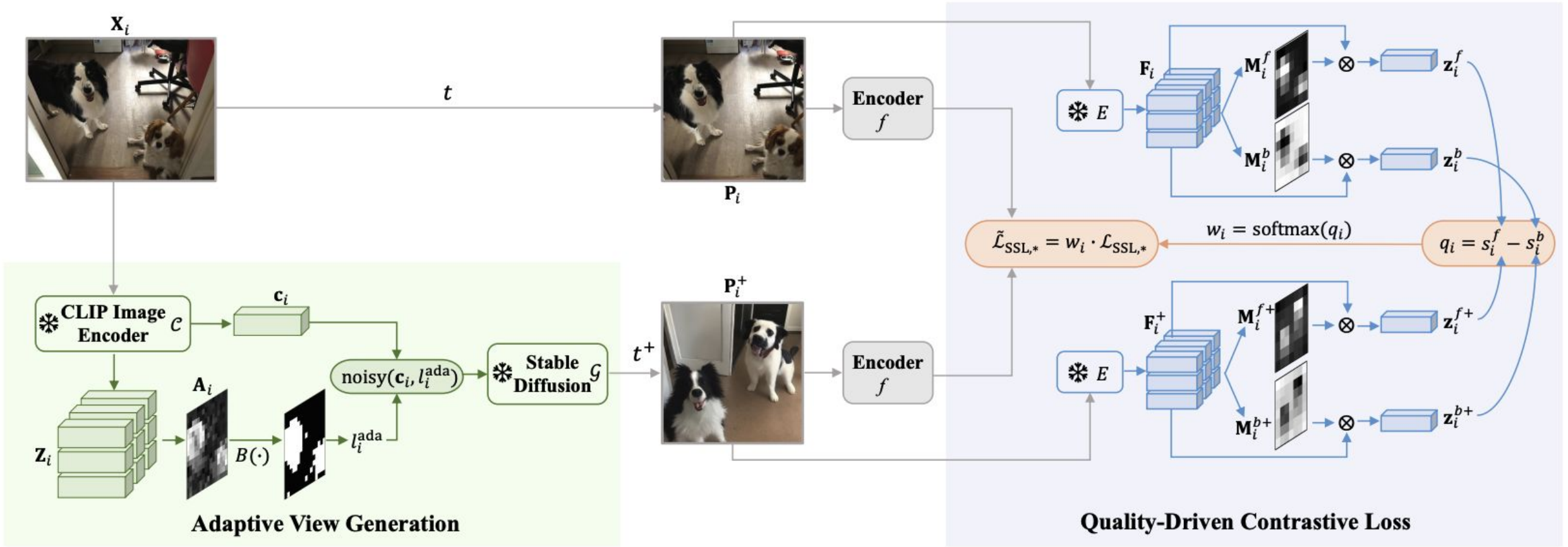
(c) Pose and background change while the object's semantics are preserved.

(d) The quality of the generated results is relatively low -> assigned a low weight of 0.8.

GenView's goal is to vary elements such as pose, viewpoint, and background while preserving object identity

GenView Framework

- GenView has two components

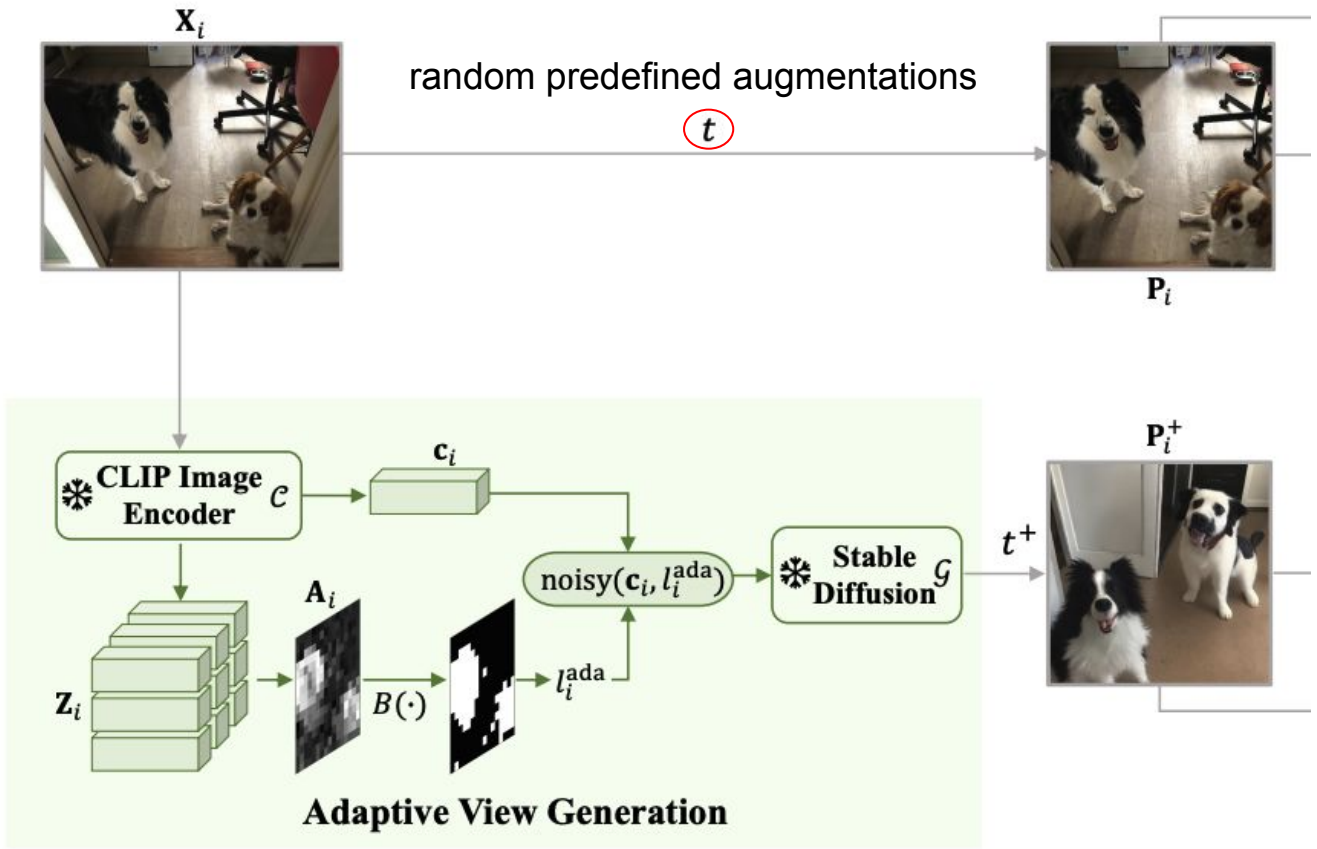


Generate a diverse positive view

Give more weight to better pairs

GenView Framework

- GenView generates two positive views



Generate Positive Views

(Up) Apply random augmentation

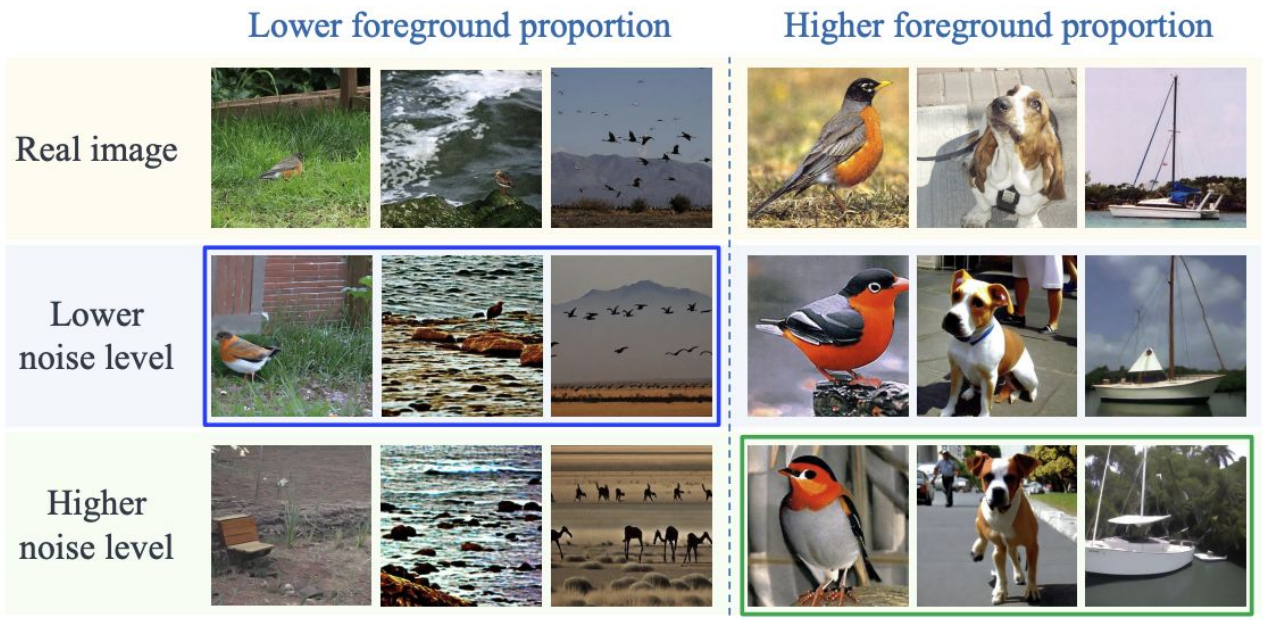
(Bottom)
- Extract CLIP image embedding
- Add adaptive noise
  - based on the foreground ratio
- Stable unCLIP generates an image

$$\mathbf{X}_i^+ = \mathcal{G}(\mathbf{z}_T, \mathbf{noisy}(\mathbf{c}_i, l_i^{\text{ada}}), w)$$

conditional image embedding perturbation strength

Adaptive View Generation

- How much variation should we introduce?



Small foreground

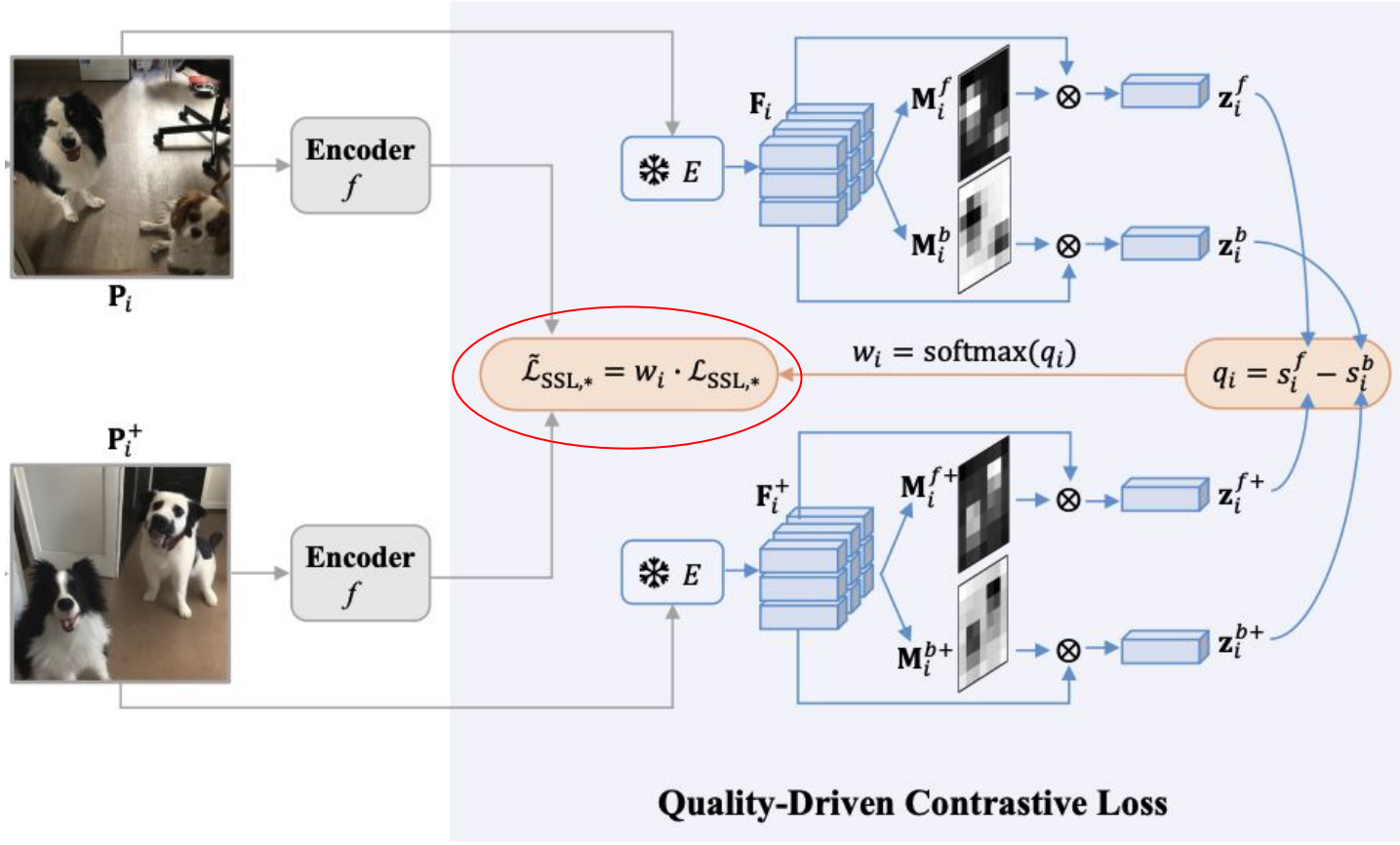
Use less noise.
The object may easily disappear.

Large foreground

Use more noise.
More variation can be introduced safely.

GenView Framework

- Apply SSL loss

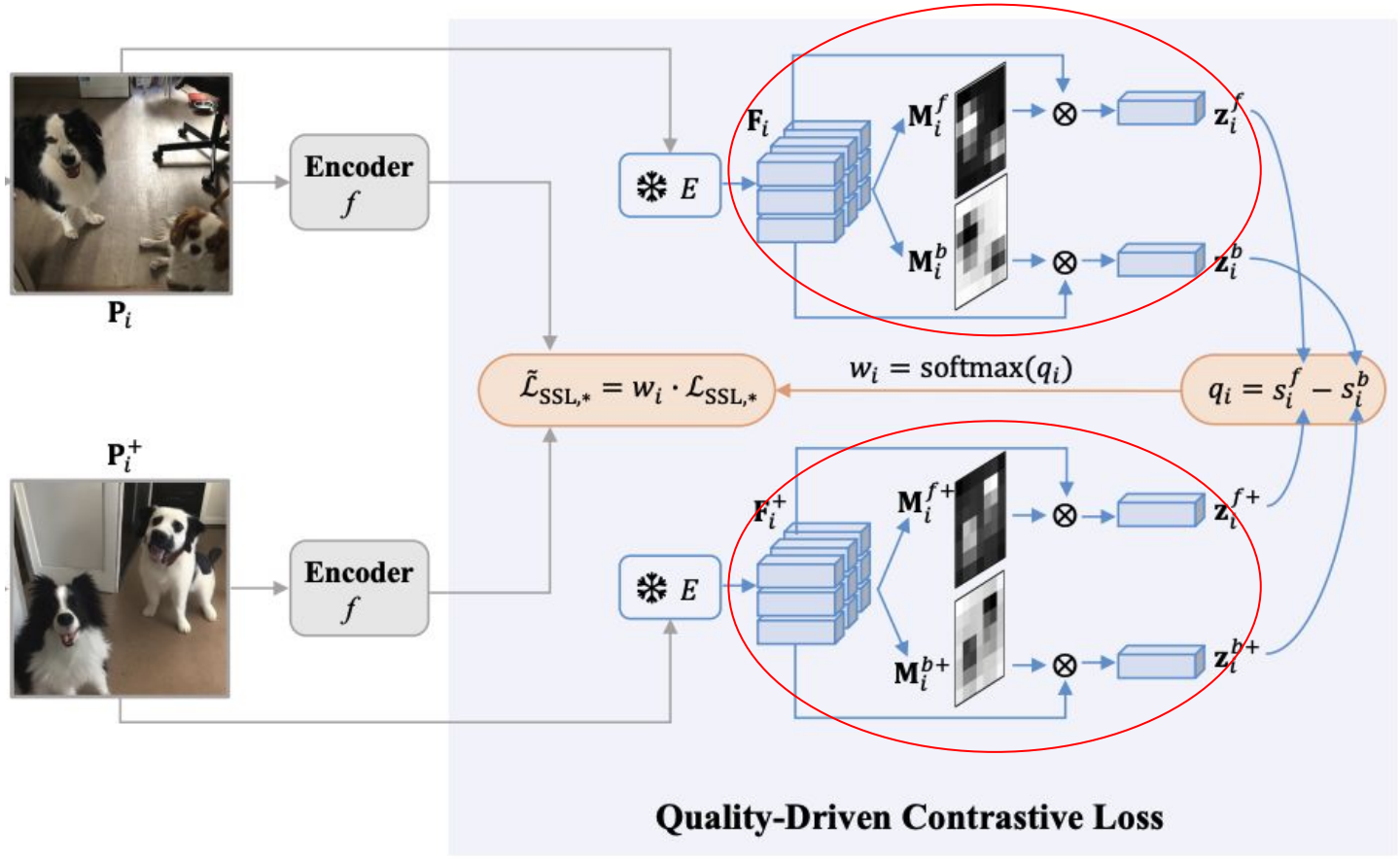


Train the SSL Encoder

- Input the two positive views into the target encoder
- Calculate the base SSL loss using the two representations:
  - Various loss functions can be used

GenView Framework

- Evaluate Pair Quality



A separate frozen encoder evaluates the quality of the pair

- First, extract the feature maps of the two views
- Next, divide them into foreground and background, respectively
- Calculate the similarity for the foreground and background, between the two views.

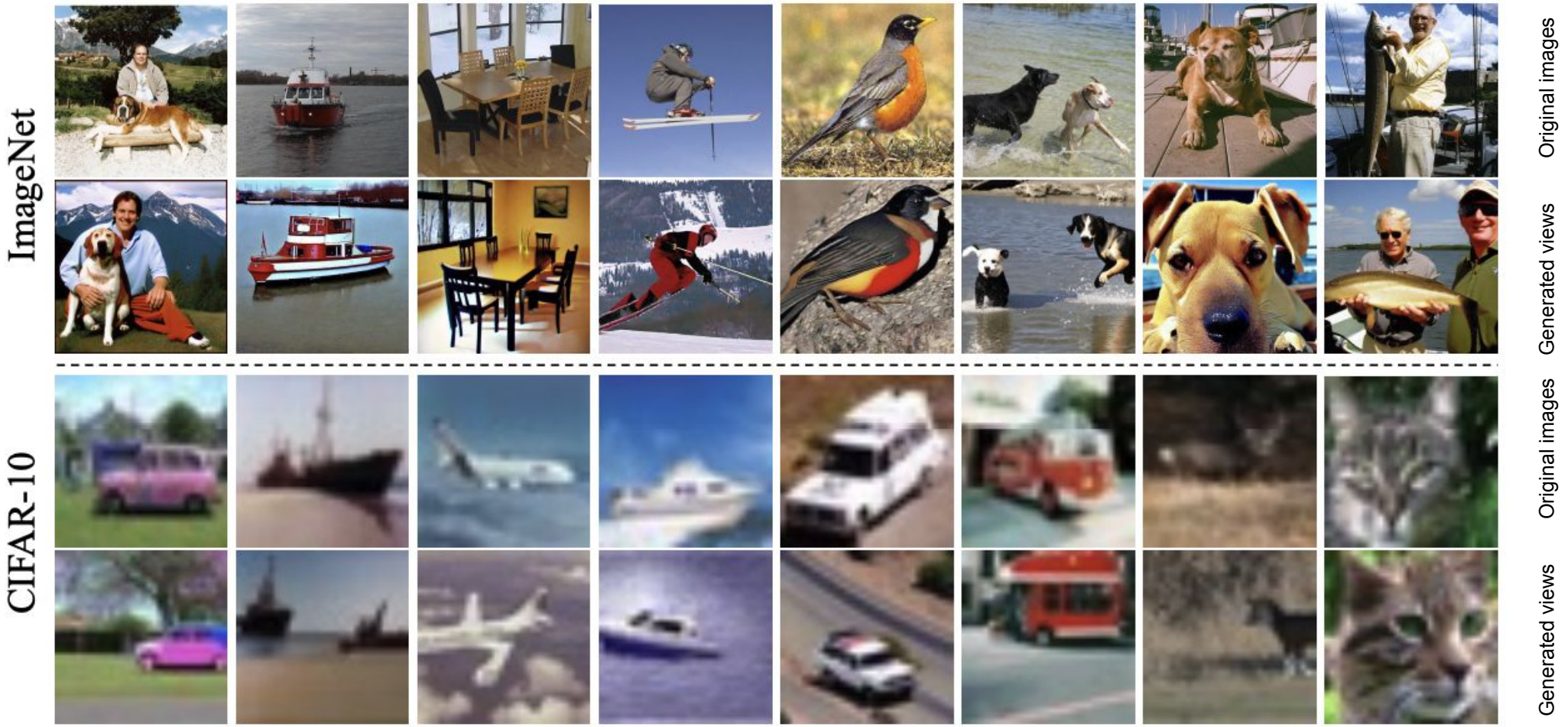
The conditions for a good positive pair are as follows:

- Similar foreground
- Different background

Give more weight to high-quality positive pairs

$$q_i = s_i^f - s_i^b$$

What Does GenView Generate?



GenView changes:
- Background
- Object pose
- Viewpoint

The main semantics remain unchanged.

Does Better View Construction Help?

- ImageNet-1K linear evaluation with ResNet-50

Table 1: Linear evaluation on IN-1K.
*: our reproduction.

Method	Architecture	Epochs	Top-1
InstDisc [94]	ResNet-50	200	56.5
SimCLR [16]	ResNet-50	200	66.8
PCL [47]	ResNet-50	200	67.6
Adco [67]	ResNet-50	200	68.6
InfoMin [84]	ResNet-50	200	70.1
NNCLR [23]	ResNet-50	200	70.7
LEVEL [39]	ResNet-50	200	72.8
Barlow Twins [102]	ResNet-50	300	71.4
CLIP [68]	ResNet-50	-	74.3
MoCov2 [33]	ResNet-50	200	67.5
MoCov2 + C-Crop [66]	ResNet-50	200	67.8
MoCov2 + GenView	ResNet-50	200	**70.0**
SwAV [12]*	ResNet-50	200	70.5
SwAV + GenView	ResNet-50	200	**71.7**
SimSiam [18]	ResNet-50	200	70.0
SimSiam + GenView	ResNet-50	200	**72.2**
BYOL [30]*	ResNet-50	200	71.8
BYOL + GenView	ResNet-50	200	**73.2**
MoCov3 [19]	ResNet-50	100	68.9
MoCov3 + GenView	ResNet-50	100	**72.7**
MoCov3 [19]	ResNet-50	300	72.8
MoCov3 + GenView	ResNet-50	300	**74.8**
MoCov3 [19]	ViT-S	300	73.2
MoCov3 + GenView	ViT-S	300	**74.5**
MoCov3 [19]	ViT-B	300	76.7
MoCov3 + GenView	ViT-B	300	**77.8**

GenView improves different SSL frameworks.

- The benefit comes from better pair construction,
- not simply from adding more images.

Contrastive learning depends on the quality of its positive views.

Limitations

- It relies on prior knowledge from Frozen CLIP and Stable unCLIP.
  - Therefore, it is **not a fully independent SSL** method trained solely on the target dataset.
- While image generation is performed offline, it **requires additional computation and storage space**.
- Determining noise levels based on the foreground ratio is a relatively simple **heuristic**.
- Quality weighting mitigates the impact of semantic drift but **does not eliminate it entirely**.

Even with good positive pairs, does contrastive learning preserve every useful feature?



Learning the Unlearned: Mitigating Feature Suppression in Contrastive Learning

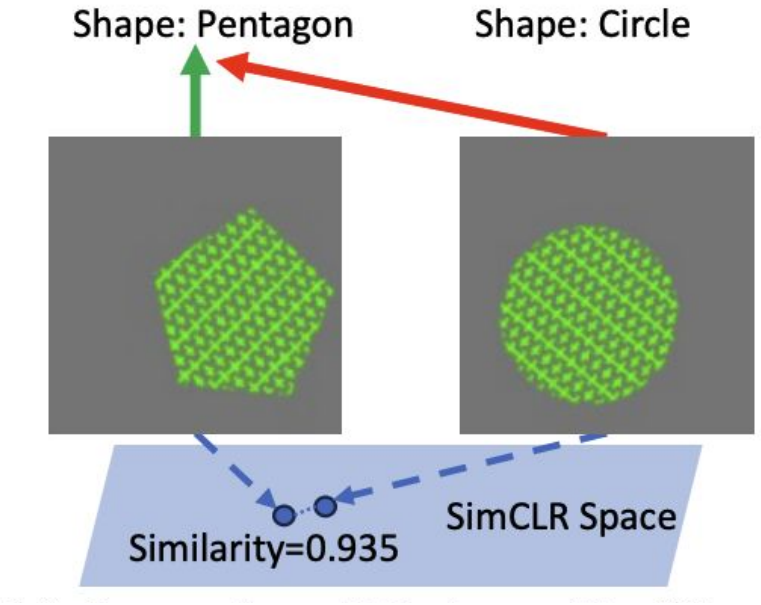
Jihai Zhang[1], Xiang Lan[2], Xiaoye Qu[3], Yu Cheng[1], Mengling Feng[2(✉)], and Bryan Hooi[2(✉)]

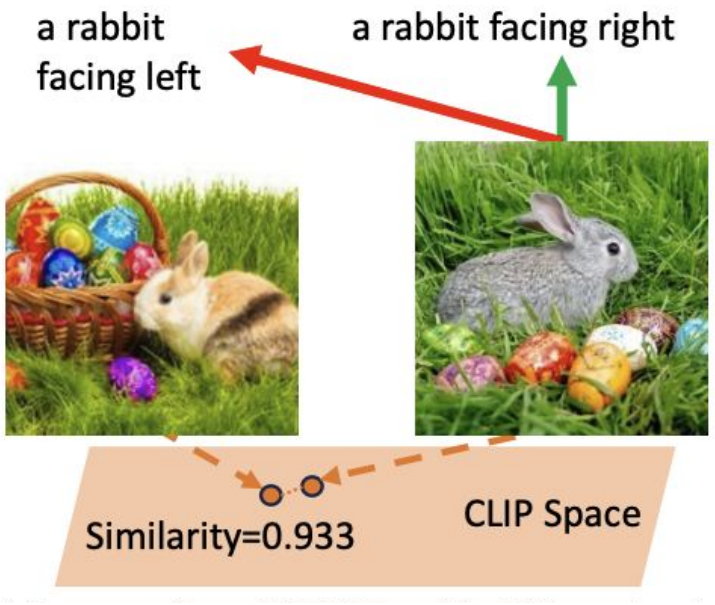
[1] The Chinese University of Hong Kong, Sha Tin, Hong Kong
[2] National University of Singapore, Singapore, Singapore
ephfm@nus.edu.sg, bhooi@comp.nus.edu.sg
[3] Shanghai AI Laboratory, Shanghai, China

Zhang et al., ECCV 2024, Learning the Unlearned: Mitigating Feature Suppression in Contrastive Learning

Feature Suppression

- The model learns some features but ignores other useful features





SimCLR

- **Same color and texture**
- **Different shape**
- **Cosine similarity: 0.935**

CLIP

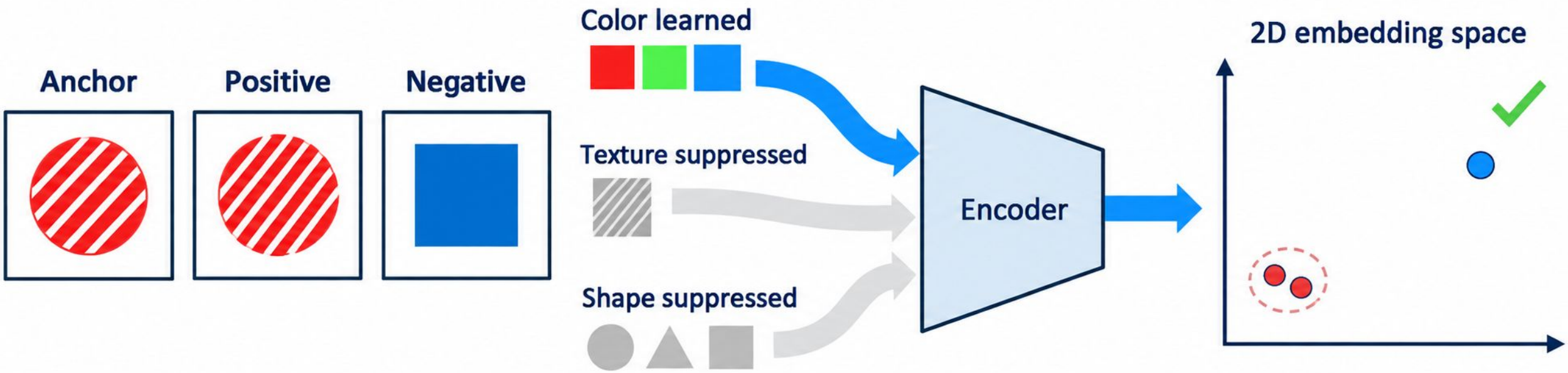
- **Same object**
- **Different orientations**
- **Cosine similarity: 0.933**

High similarity does not always mean the same semantics.

Feature Suppression

- Although the two representations exhibit high similarity, the information crucial for downstream tasks differs.
  - (previous slide) SimCLR focuses on color and texture while ignoring shape
  - (previous slide) CLIP recognizes the object ‘rabbit’ but struggles to distinguish its orientation
- This phenomenon, where useful information present in the input is lost in the representation, is known as feature suppression.

Why Does Feature Suppression Happen?

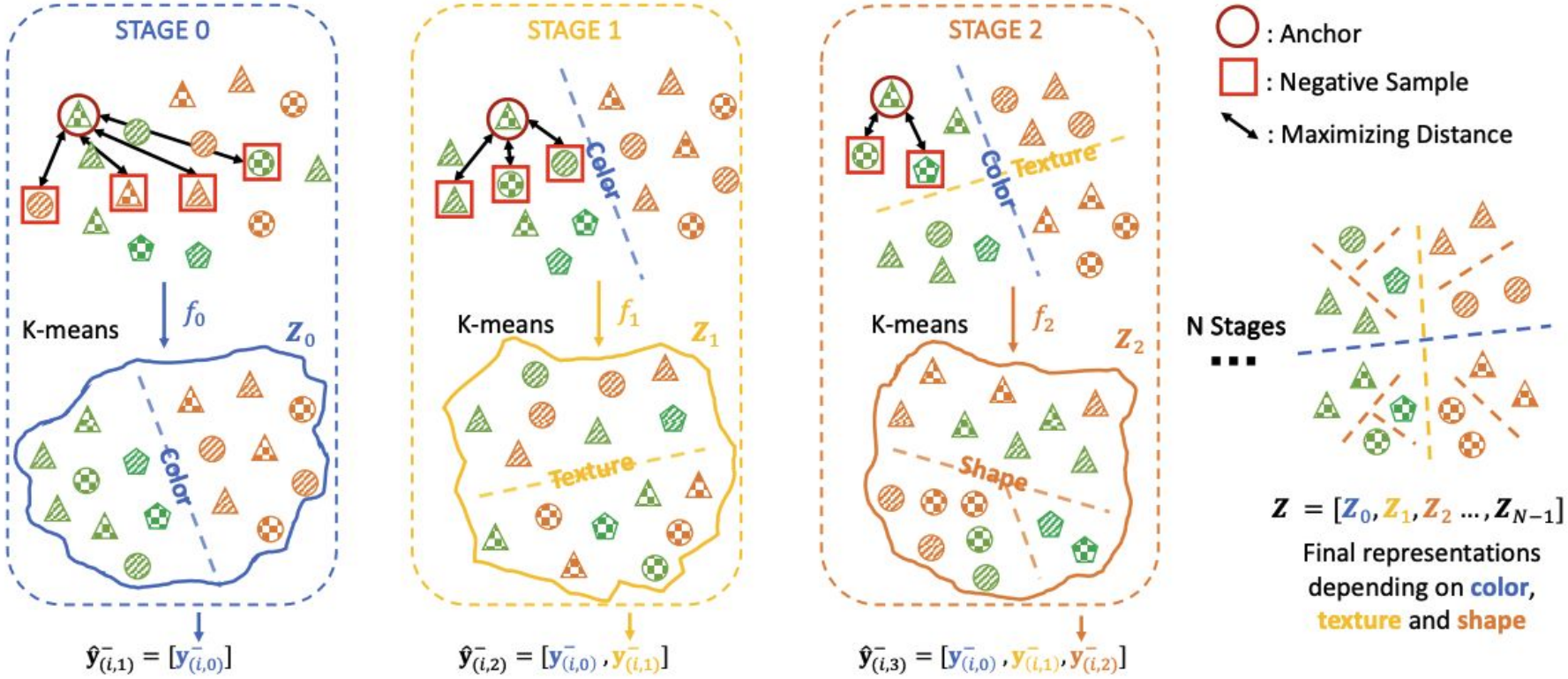


- Both Anchor and Positive are represented by red striped circles, while Negative is a blue square.
- The model can easily distinguish between positive and negative samples using color alone.
- The contrastive objective is already satisfied.
- Therefore, there is no need to additionally learn texture and shape.

The model learns the easiest discriminative feature (Color), while other useful features are suppressed.

Multistage Contrastive Learning

- MCL learns different features in multiple stages.

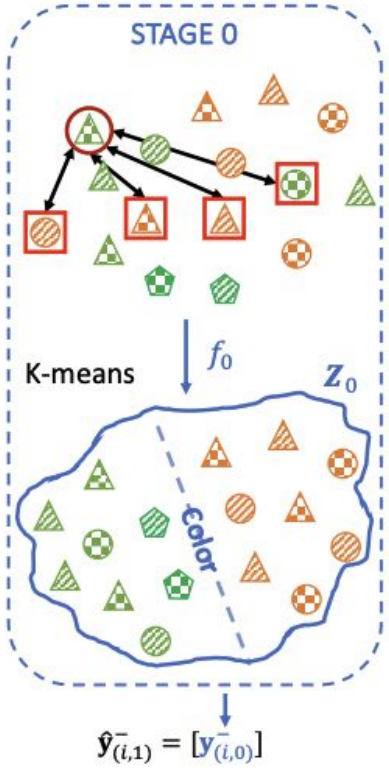


① Train
② Cluster
③ Select Negatives Within Clusters
④ Repeat
⑤ Concatenate Representations

Each stage removes the shortcut used in previous stages.

Multistage Contrastive Learning

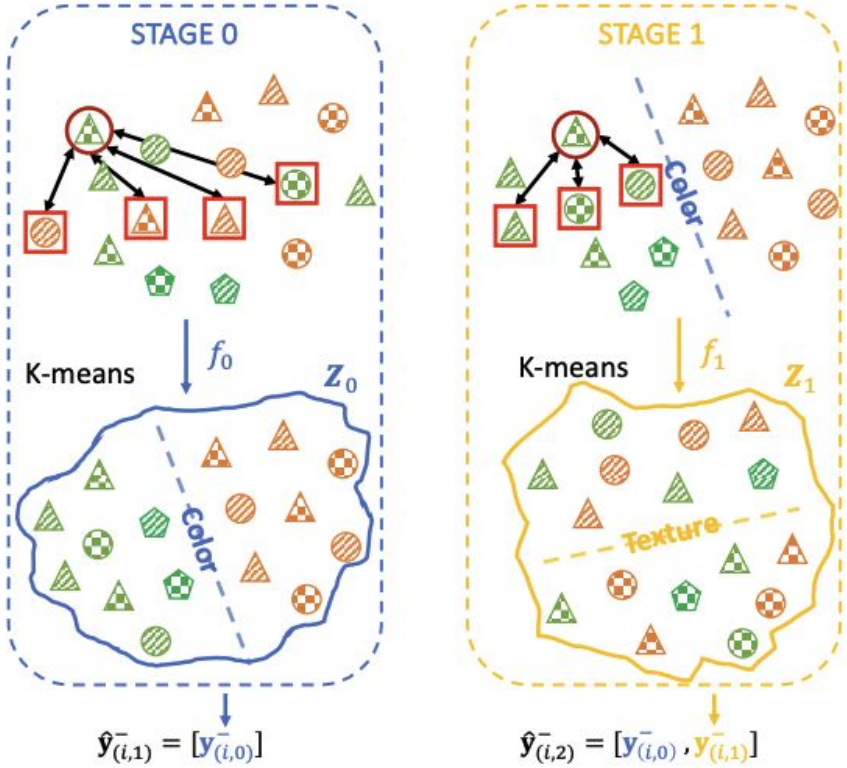
- MCL learns different features in multiple stages.



Stage 0
- Standard contrastive learning is performed.
  - It is assumed that the model first learns color, which is the simplest feature.
  - As a result, the representation space is partitioned based on color.
- Clustering
  - K-means clustering is applied to the representations from Stage 0.
  - Samples within the same cluster share similar dominant features learned in the previous stage.

Multistage Contrastive Learning

- MCL learns different features in multiple stages.

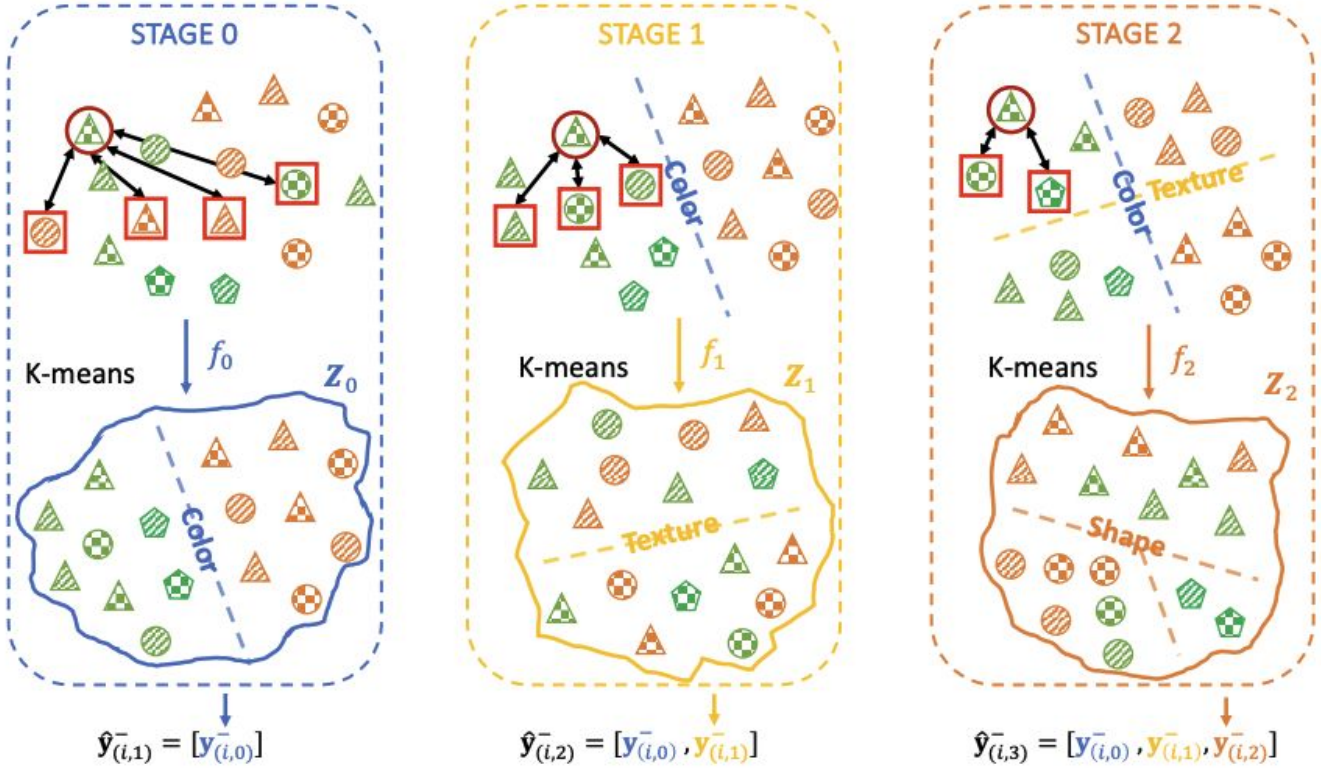


Stage 1

- Negative samples are selected only from the same cluster as the anchor.
  - Anchor: green textured triangle
  - Negatives: other green objects
- Since all negatives share the same color, color can no longer be used to distinguish between them.
  - Consequently, the model must learn other features, such as texture.

Multistage Contrastive Learning

- MCL learns different features in multiple stages.

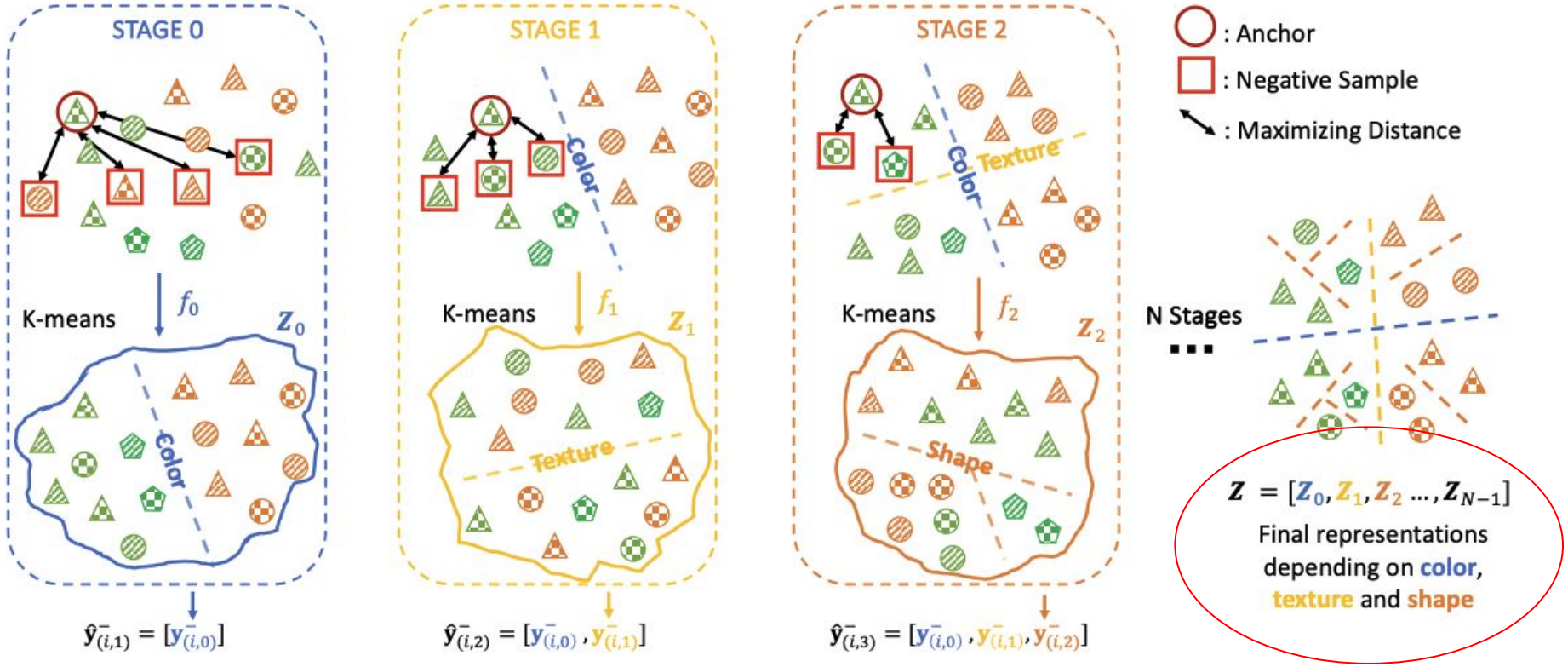


Stage 2

- Cluster assignments from both Stage 0 and Stage 1 are used together.
  - Same-color cluster
  - Same-texture cluster
- Negative samples are selected from within the group that satisfies these conditions.
  - Since neither color nor texture can be used for discrimination in this case, the model learns to focus on shape.

Multistage Contrastive Learning

- MCL learns different features in multiple stages.



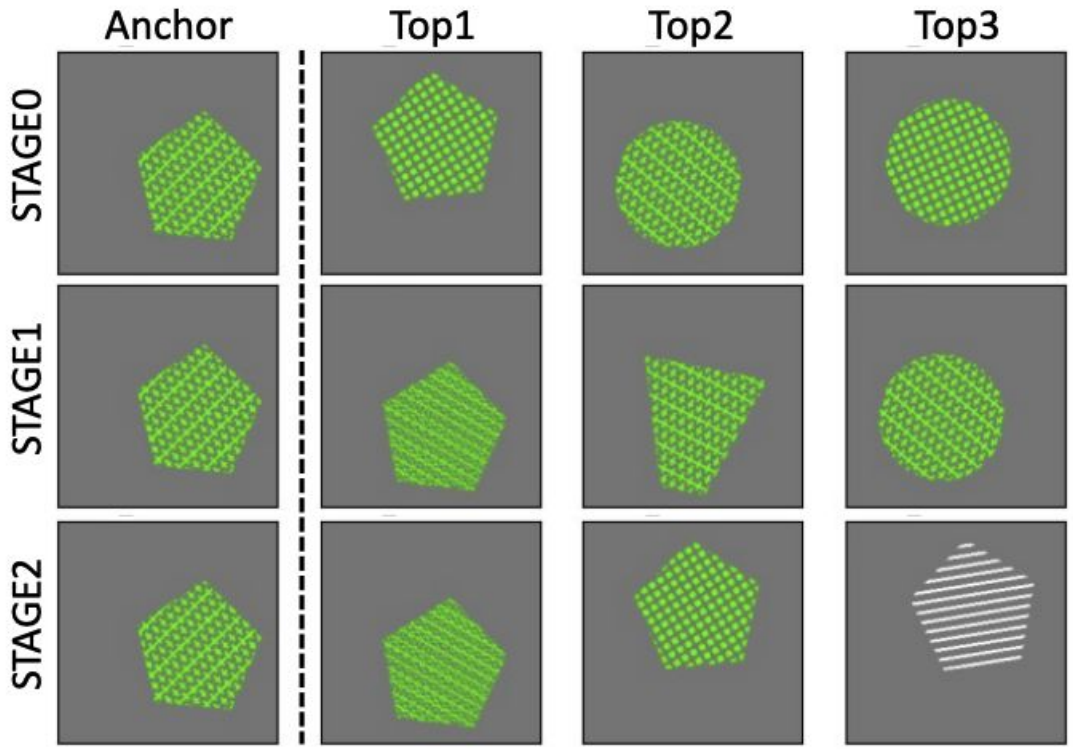
Concatenation preserves features learned at all stages.

Difference from Hard Negative Mining

- Hard negative mining: Selects negatives that are close to the current embedding.
- MCL: Selects only negatives that belong to the same cluster from the previous stage.
- Objective: Rather than finding difficult samples, the goal is to prevent the reuse of previously learned features.
- In other words, while MCL may appear similar to hard negative mining, its objective is different.

What Does Each Stage Learn?

- The nearest samples reveal what each stage has learned.



Stage 0: Samples of the same color as the anchor are nearby
Stage 1: Samples with the same texture are nearby
Stage 2: Samples with the same shape are nearby

MCL changes the dominant feature across stages.

Results on Suppressed Features

- MCL improves suppressed features without losing previously learned features.

Table 2: Linear evaluation accuracy of MCL and the baselines. Bold indicates the best performance.

	Trifeature			CelebA			CIFAR-MNIST		
	Shape	Texture	Color	Hair	Smiling	Gender	CIFAR	MNIST	Average
IFM	0.99	0.99	**1.00**	0.82	0.70	0.93	0.11	**0.99**	0.82
TS	0.93	**1.00**	**1.00**	0.62	0.65	0.87	0.09	**0.99**	0.77
FD	0.75	0.73	0.78	0.81	**0.89**	0.94	0.78	0.86	0.82
SimCLR	0.81	0.99	**1.00**	0.84	0.75	0.94	0.29	0.98	0.83
MCL	**1.00**	**1.00**	**1.00**	**0.85**	0.79	**0.95**	**0.87**	**0.99**	**0.93**

Multimodal Feature Suppression

Table 3: Accuracy of the MCL tuned CLIP models on the MMVP benchmark. Bold indicates the best performance across four stages. The STAGE0 model is the original OpenAI pre-trained model. The final MCL model result is obtained by using the concatenation of the representations across all four stages as the final representation.

	O&D	PSF	S&C	Q&C	P&R	C&A	S&P	Texts	V&P	Average
ResNet STAGE0	6.7	6.7	**46.7**	0.0	**13.3**	46.7	**33.3**	6.7	13.3	19.3
ResNet STAGE1	**13.3**	**13.3**	40.0	**20.0**	0.0	33.3	33.3	6.7	26.7	20.7
ResNet STAGE2	6.7	6.7	33.3	6.7	6.7	**73.3**	13.3	13.3	**33.3**	21.5
ResNet STAGE3	0.0	13.3	33.3	20.0	0.0	60.0	33.3	**26.7**	20.0	23.0
ResNet MCL	0.0	13.3	40.0	6.67	6.67	73.3	40.0	6.7	33.3	<u>24.4</u>
ViT STAGE0	**26.7**	13.3	26.7	6.7	6.7	40.0	26.7	**13.3**	20.0	20.0
ViT STAGE1	13.3	**33.3**	**66.7**	26.7	**13.3**	53.3	20.0	13.3	**26.7**	29.6
ViT STAGE2	0.0	13.3	46.7	**40.0**	6.7	53.3	20.0	13.3	20.0	23.7
ViT STAGE3	13.3	6.67	46.7	13.3	13.3	**66.7**	**40.0**	13.3	20.0	25.9
ViT MCL	6.67	20.0	73.3	13.3	13.3	80.0	46.7	13.3	26.7	<u>32.6</u>

Feature suppression is not an issue unique to SimCLR. While CLIP effectively represents object categories, it can overlook fine-grained details such as orientation, count, and spatial relationships. Applying MCL increased the average MMVP accuracy for ViT models from 20.0 to 32.6.

Summary

- Standard Contrastive Learning Learns the easiest discriminative features.
- MCL
  - Selects negatives with similar learned features.
- Result
  - Each stage learns previously suppressed features.

Contrastive learning may learn enough to separate samples, but not enough to preserve all useful information.

Limitation

- Training multiple stages and encoders increases the **computational** load.
- Concatenating representations from each stage also **increases** the feature **dimension**.
- There is **no guarantee that K-means clusters accurately** represent actual semantic features.
- **Simply concatenating** representations from each stage may **not** be **optimal**.

Are all negative samples equally unrelated?

Published as a conference paper at ICLR 2025

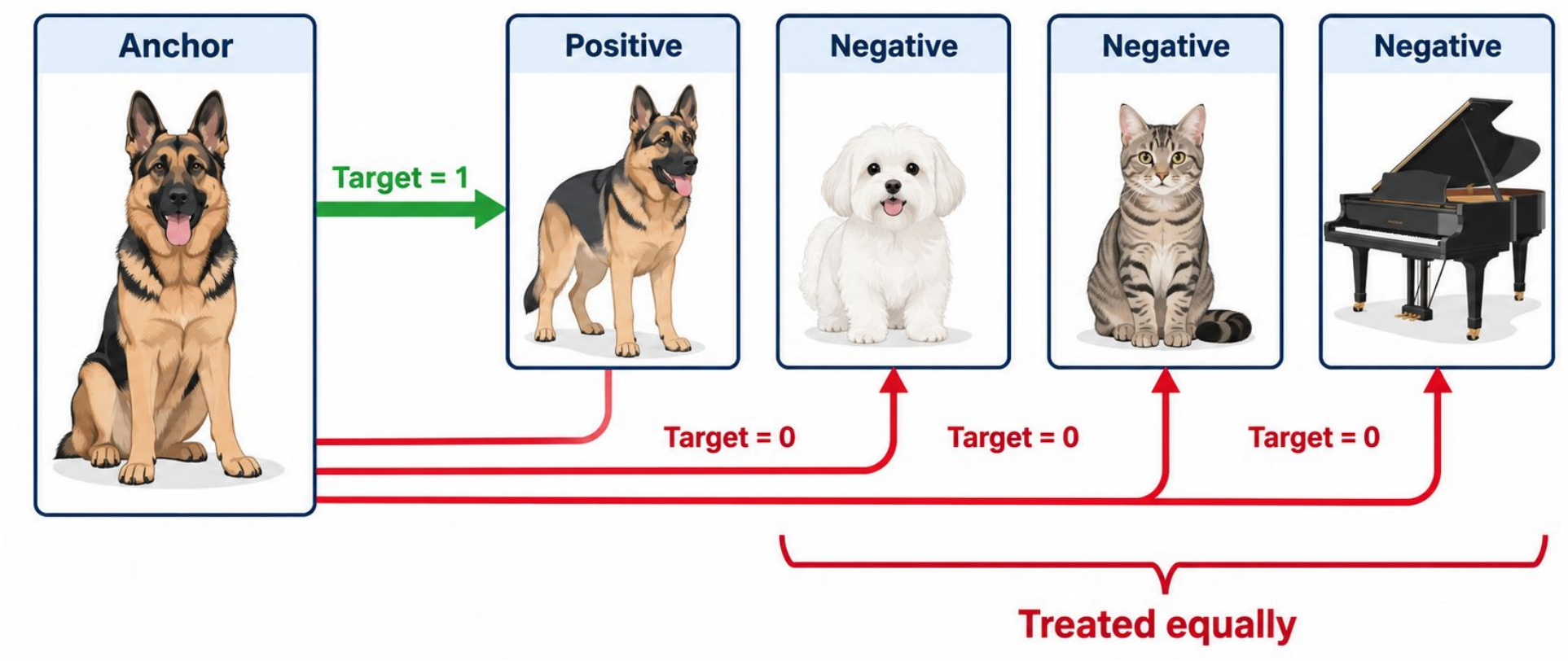
𝕏-SAMPLE CONTRASTIVE LOSS: IMPROVING CONTRASTIVE LEARNING WITH SAMPLE SIMILARITY GRAPHS

Vlad Sobal[1,2] **Mark Ibrahim**[1] **Randall Balestriero**[3] **Vivien Cabannes**[1]
Diane Bouchacourt[1] **Pietro Astolfi**[1] **Kyunghyun Cho**[2,4,5] **Yann LeCun**[1,2]
[1]Meta FAIR [2]New York Univeristy [3]Brown University [4]Genentech [5]CIFAR

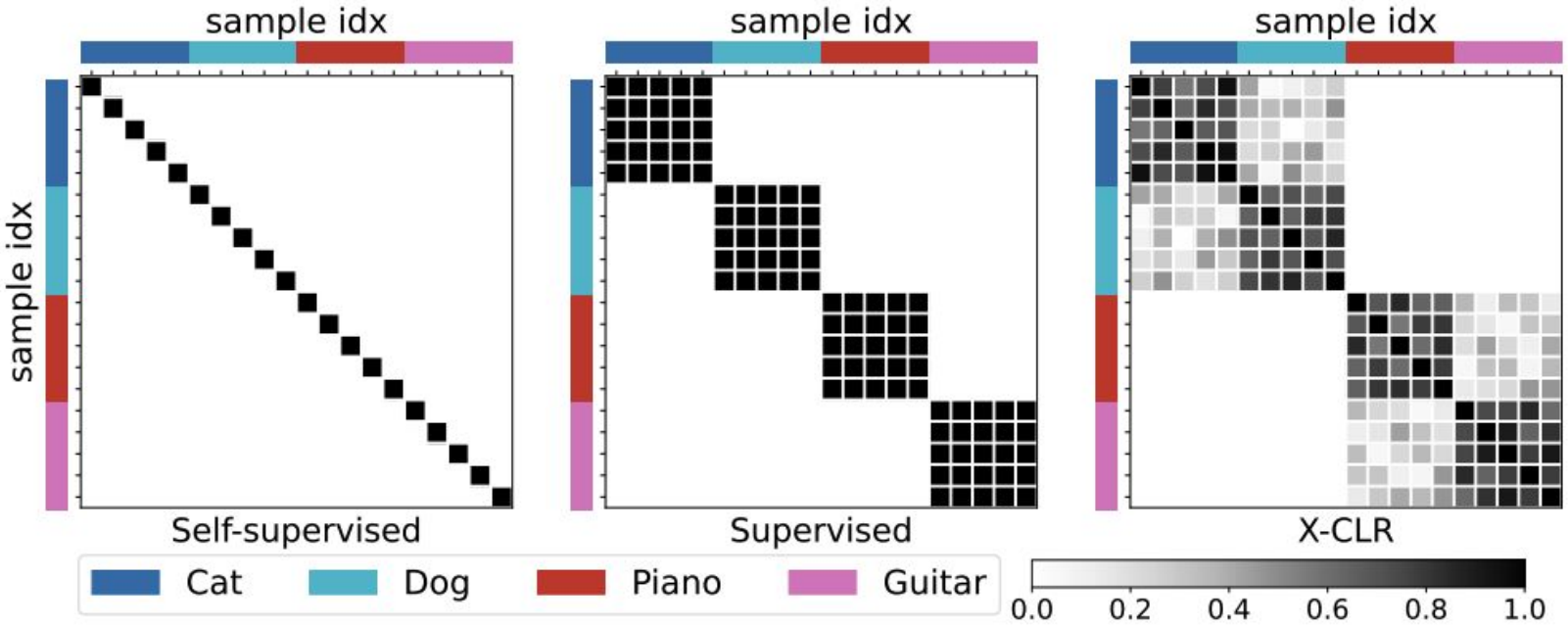
Sobal et al., ICLR 2025, X-SAMPLE CONTRASTIVE LOSS: IMPROVING CONTRASTIVE LEARNING WITH SAMPLE SIMILARITY GRAPHS

Are all negative samples equally unrelated?

- Should another dog, a cat, and a piano be treated as equally negative?



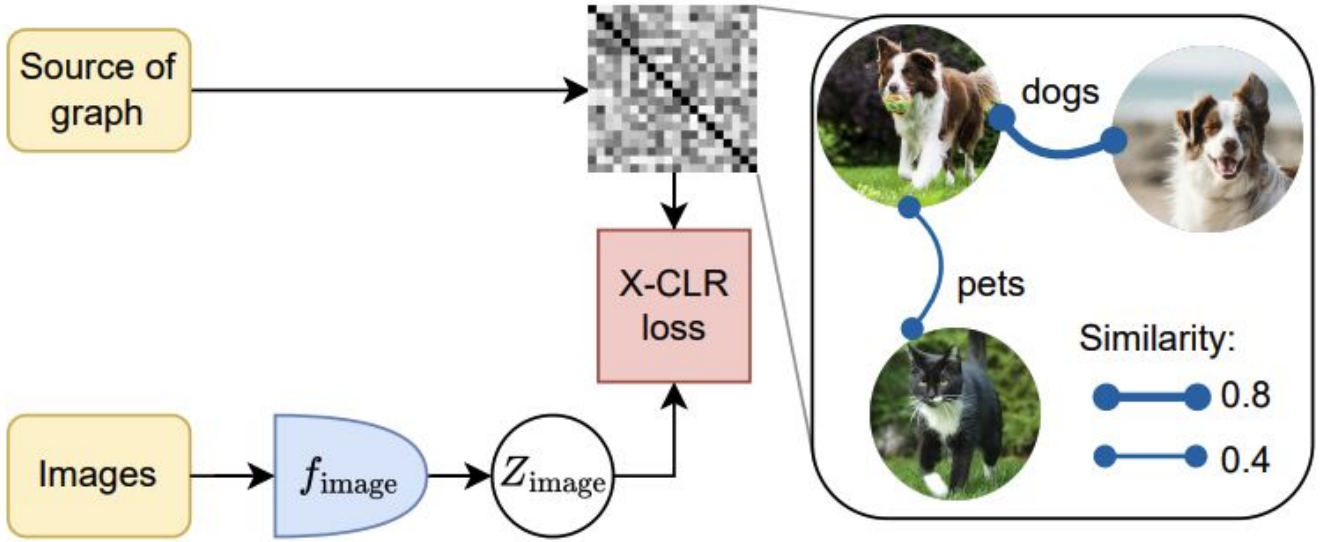
Contrastive Learning as a Similarity Graph



1. Self-supervised learning
   Only two views of the same image are connected.
2. Supervised contrastive learning
   Samples from the same class are connected.
3. X-CLR
   Samples can have different degrees of similarity.

X-CLR replaces a binary graph with a soft similarity graph.

Where Does the Similarity Graph Come From?



A similarity graph can come from:

- Text captions
- Class descriptions
- Semantic hierarchies
- Other metadata

[EX] ImageNet
German shepherd
→ “a photo of a German shepherd”
Maltese dog
→ “a photo of a Maltese dog”

- The sentence is represented by the class name.
- A fixed text encoder converts each sentence into an embedding.
- Target similarity between the two samples

If there are captions in the dataset, we can just use it.

X-CLR Objective

```
def training_step(captions, images):
    # encode captions a frozen text encoder
    txt_emb = txt_enc(captions).detach() # N x D_txt
    # encode images with the model we are training
    img_emb = img_enc(images)  # N x D_img

    # calculate similarities
    tgt_sims =
        softmax(txt_emb @ txt_emb.T / tao_s, dim=1)
    img_sims =
        softmax(img_emb @ img_emb.T / tao, dim=1)
    # the loss is the cross-entropy between the two
    return CE(tgt_sims, img_sims, dim=1).mean()
```

Target graph
- Similarity from text or metadata

Learned graph
- Similarity between image representations

Training objective
- Match the learned graph to the target graph

$$\mathcal{L}_{\mathbb{X}\text{-CLR}} = \frac{1}{2N_b} \sum_{i=1}^{2N_b} H(s_i, p_i)$$

$$p_{i,j} = \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

$$s_{i,j} = \frac{\exp(\boldsymbol{G}_{i,j}^{(\text{soft})}/\tau_s)}{\sum_{k=1}^{2N_b} \exp(\boldsymbol{G}_{i,k}^{(\text{soft})}/\tau_s)}$$

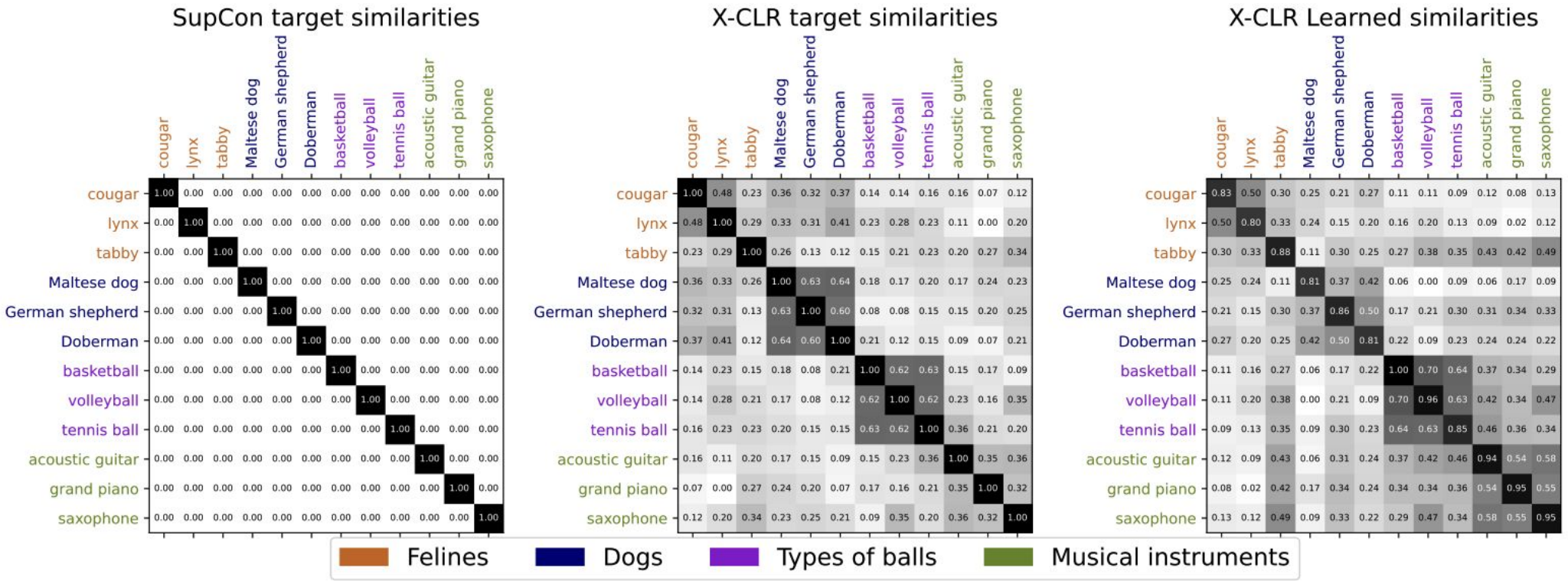
$$\boldsymbol{G}_{i,j}^{(\text{soft})} = \text{sim}(f_{\text{text}}(c_i), f_{\text{text}}(c_j))$$

The Role of Similarity Temperature

- τ: This is a common contrastive temperature used for image representation similarity.
- τ_s: It determines how "soft" to make the target similarity graph.
  - Low target temperature
    - Focus on the strongest positive.
    - Similar to standard contrastive learning.
  - High target temperature
    - Give more weight to related samples.
    - The target graph becomes softer.

The graph should preserve the true positive while including soft positives.

What Does X-CLR Learn?



The embedding space reflects semantic structure beyond class labels.

Does the Soft Graph Help?

Table 1: **Accuracy of 𝕏-CLR and a range of baselines on a set of classification benchmarks.** We freeze the encoder, and train a linear layer on top to probe the quality of representations. We test on ImageNet classification, as well as on a set of robustness benchmarks.

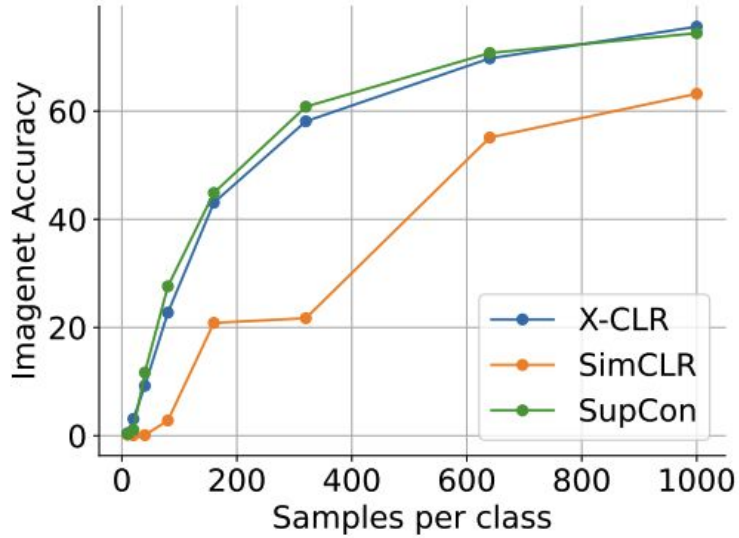
Method	Soft loss	Uses labels	ImageNet	ImageNet Real	Background Decomp.		ObjectNet	MIT States	
					Same Class	Mixed		Objects	Attributes
SCE	✓	✗	71.3	78.7	61.7	58.4	20.2	44.5	31.0
ReSSL (1 crop)	✓	✗	69.4	76.9	56.3	53.2	18.3	44.5	31.2
VICReg	✗	✗	72.4	79.0	60.8	56.8	20.5	43.5	26.9
Barlow Twins	✗	✗	72.8	80.0	62.7	59.4	21.6	**45.9**	**31.7**
SimCLR	✗	✗	63.4	67.8	44.7	38.9	12.1	40.9	29.1
SupCon	✗	✓	74.3	79.7	64.0	59.1	24.4	45.6	30.8
𝕏-CLR	✓	✓	**75.6**	**81.5**	**66.6**	**62.7**	**27.5**	**45.9**	31.1

X-CLR improves classification and background robustness.

X-CLR in Low-Data Settings

Table 3: **𝕏-Sample Contrastive learning performance with CC3M and CC12M training.** We train 𝕏-CLR on conceptual caption images with text caption similarities, and compare to CLIP and SimCLR. SimCLR does not use text data and is only trained on images.

				Background Decomposition		
Data	Method	ImageNet	ImageNet Real	Same Class	Mixed	ObjectNet
CC3M	SimCLR	57.0	64.0	24.4	18.9	10.8
	CLIP	41.0	47.6	12.5	10.6	7.8
	𝕏-CLR	**58.2**	**65.6**	**26.7**	**20.3**	**11.5**
CC12M	SimCLR	58.9	66	24.6	19.8	12.7
	CLIP	58.8	66.1	20.5	17.1	11.9
	𝕏-CLR	**59.4**	**66.7**	**26.1**	**20.4**	**13.4**



Soft relationships are especially useful when training data is limited.

Limitations

- Captions, labels, or **metadata** are **required** to construct the similarity graph.
- Low **graph quality** leads to poor representation quality.
- **Noisy text**, such as CC captions, may fail to provide accurate semantic relations.
- **Storing** a graph of all samples requires significant **memory**; therefore, practical implementations necessitate batch-level similarity calculations or pre-computation.
- **Biases inherent** in the **text encoder** can also be propagated to the target graph.

Three Views of Better Contrastive Learning

Paper	Main question	Solution
GenView	Are positive views good enough?	Generate and weight better positive views
MCL	Are useful features suppressed?	Learn different features across stages
X-CLR	Are sample relations binary?	Learn from a soft similarity graph

Contrastive learning depends on the views, the features, and the relationships used for training.

Next Week..

- Continue on Contrastive Learning ..